Engineering Applications of Artificial Intelligence 174 (2026) 114451

Contents lists available at ScienceDirect

Engineering Applications of Artificial Intelligence

journal homepage: www.elsevier.com/locate/engappai

Root cause analysis of cloud platform faults based on anomaly correlation graph and graph neural network

Beijing Zhou [a], Bingyan Chen [a,1], Li Yan [a,*], Xinyu Liang [a], Zongmin Ma [a,b,**]

[a] College of Computer Science & Technology, Nanjing University of Aeronautics and Astronautics, China
[b] Collaborative Innovation Centre of Novel Software Technology and Industrialization, China

ARTICLE INFO

Keywords:
Alarm root cause analysis
Anomaly correlation graph
Graph attention neural network
Light gradient boosting machine

ABSTRACT

The increasingly complex structure of the cloud environment makes it more difficult to deal with faults. Fault root cause analysis can locate the root cause of fault and promote the troubleshooting process. Traditional fault root cause analysis usually relies on the experienced judgment of engineers, which is time-consuming and poor in scalability. This paper proposes a novel model to solve the problem of fault root cause analysis in cloud platforms with complex network topology and massive alarms generation. Our model combines anomaly correlation graph, graph attention network and light gradient boosting machine. The causal correlations are first learned from historical alarm data to construct the anomaly correlation graph. To reduce noise interference, we propose a weight threshold sampling mechanism to select neighbor nodes with high causal correlation as the aggregation targets of the central node. Graph attention network is then used to aggregate neighborhood information to achieve node feature extraction, which can effectively represent the complex dependencies among nodes. Finally, light gradient boosting machine is applied for classification to realize root cause localization. Moreover, interpretable reports are generated through similarity analysis to provide references for engineers in repairing faults. The model can locate the root cause without excessive mass training samples. The experimental results show that our proposed model can achieve better effect of root cause analysis than other relevant methods, where the precision is increased by more than 10%.

1. Introduction

With the advent of the cloud computing era and the rapid development of network information technology, the scale of cloud platforms continues to expand. Faced with more devices, applications, and connections in the system, traditional manual O&M (operations and maintenance) is no longer able to cope with such a complex IT (Information Technology) environment (Notaro et al., 2021). To meet the current needs of large-scale and highly complex system operation and maintenance, AIOps (Artificial Intelligence for IT Operations) comes into being. AIOPs refers to the operation and maintenance method that relies on artificial intelligence (AI) technologies such as machine learning to observe the operation status of infrastructure, automatically learn and summarize rules from massive operation and maintenance data, and make decisions (Notaro et al., 2021). AIOPs helps improve engineering efficiency and system reliability (Shen et al., 2020).

In recent years, with the increasingly mature monitoring capabilities of cloud platforms and the increasing complexity of systems in the cloud environments, the amount of data generated has increased dramatically. An important issue is how to establish an appropriate mechanism to determine the root cause of failure, so as to process large amounts of data or provide valuable real-time feedback. Fault root cause analysis is an important part of AIOPs, which aims to locate the faulty components and find the root cause of the fault through tracing inference, based on the alarm logs and the topological connection relationship between components.

Alarm logs are one of the main monitoring data in O&M scenarios, conveying abnormal information of systems or components in a serialized form. They are an important data basis for Root Cause Analysis (RCA) of faults (Jiang and Bai, 2023). An alarm is a notification issued when an abnormal event occurs. Abnormal events refer to a variety of anormal conditions that occur in the cloud platform. An abnormal event

* Corresponding author.
** Corresponding author. Nanjing University of Aeronautics and Astronautics, Nanjing, 211106, China.
E-mail addresses: yanli@nuaa.edu.cn (L. Yan), zongminma@nuaa.edu.cn (Z. Ma).
[1] Co-first author.

https://doi.org/10.1016/j.engappai.2026.114451
Received 23 March 2025; Received in revised form 11 February 2026; Accepted 8 March 2026
Available online 20 March 2026
0952-1976/© 2026 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

corresponds to an alarm instance, such as "port 80 communication anomaly", "the request delay exceeds 5 min" and etc. In cloud environments, the generation of alarms is influenced by the topology network behind them (Cai et al., 2022). The nodes in the topology network represent the various components in the cloud platform, while the edges represent the dependency relationships between the components. A single fault in a cloud platform usually triggers a large number of different types of alarms on multiple connected nodes, leading to alarm flooding (Alinezhad et al., 2022). Basically, we can category the generated alarms into *root cause alarms*, *derived alarms*, and *noise alarms*. The derived alarm refers to the alarm that has a strong causal correlation with the alarm generated by the parent node. Noise alarm refers to the alarm that has no causal correlation or weak causal correlation with the alarm generated by the parent node. Here, "causal correlation" refers to a relationship between two events where not only is there a statistical association, but this association also has characteristics of a causal relationship, meaning one event (cause) leads to another event (effect) occurring. Root cause alarm refers to the alarm that causes the fault. The goal of RCA is to find root cause alarms and root cause nodes.

There are three major difficulties in RCA. First, complex component or service interactions in cloud platforms pose challenges to RCA (Soldani and Brogi, 2022). Second, for many alarm events, noise alarm interference distracts the operators' attention from root alarm (Alinezhad et al., 2022). Third, it is difficult to obtain interpretable RCA results for troubleshooting (Böhmer and Rinderle-Ma, 2020). How to solve these problems is the main focus of current RCA research. There have been many efforts in proposing AI-based RCA methods such as *unsupervised random walk method* (Li et al., 2022a; Ma et al., 2020; Zhang et al., 2020), *supervised traditional machine learning method* (Gaita et al., 2022; Dehbozorgi et al., 2020), and *deep learning method* (He and Zhao, 2020; Yan et al., 2022). The existing methods usually cannot solve the problem comprehensively for several limitations.

(1) Many studies mainly focus on the correlation between alarms, ignoring the causal relationship between alarms (Jiang and Bai, 2023). For studies that focus on causality, the topological dependencies between alarms are often overlooked (Cai et al., 2022).
(2) They cannot represent topological relationships between nodes in complex networks well (Li et al., 2022b). Existing methods rarely use topology to effectively extract node features, and usually ignore topological spatial information, resulting in a failure to capture the information transfer between nodes.
(3) They lack the judgment of noise alarms. In large-scale systems, alarm floods often occur, and these alarms possibly contain a lot of noise data also (Alinezhad et al., 2022). Existing work rarely makes judgments on noise data, and this affects the search for root causes and lead to inaccurate results.
(4) They lack interpretability for localization results. Few existing methods provide interpretability analysis of RCA. Many existing models are black-box models (Gaita et al., 2022; He and Zhao, 2020; Yan et al., 2022) and it is difficult for engineers to trust the localization results (Li et al., 2022c).

In this paper, we propose a fault root cause analysis model named ACG-LGraphBM, which combines Anomaly Correlation Graph (ACG), Graph Attention Network (GAT), and Light Gradient Boosting Machine (LGBM). Considering that root cause alarms usually account for only a minority of all alarms and are mostly recurring failure modes (Li et al., 2022a), in this paper, we only focus on supervised learning methods based on historical data. Unlike traditional data-driven RCA methods that typically ignore the causal relationships among abnormal alarms, the ACG-LGraphBM model learns the causal structure of abnormal alarm event sequences, constructs the ACG, and calculates the correlation degree, which refers to a quantified measure of the association between abnormal alarm events. Particularly, we propose the time-delay mechanism to deal with possible delays in the collection of certain alerts for the ACG construction. To use the topological spatial information and reduce the noise interference, a weight threshold sampling mechanism is proposed to sample the neighbor nodes. The neighborhood information of the central node is aggregated by GAT and the multi-head attention mechanism is introduced to strengthen the representation of features. Due to the better performance of traditional machine learning methods in training stable classification models (Deng et al., 2021), we select the LGBM classifier for further feature learning and classification. Finally, a root cause node sorting list and root cause alarms are obtained, containing the probability that each node as the root cause. The use of time-delay mechanism and weight threshold sampling mechanism can improve the accuracy of root cause analysis. In addition, we propose an interpretability module via similarity analysis of nearest-neighbor historical cases, generating interpretable reports for engineers' reference. The experimental results show that the proposed ACG-LGraphBM has good accuracy in root cause localization. The main contributions of this paper are summarized as follows.

(1) The Time-delay Topological Hawkes Process (TTHP) is proposed to learn the causal structure and construct the ACG, where the topological dependences behind the abnormal alarm event sequences are considered.
(2) The attention-based graph neural network is applied for node feature extraction. We propose a weight threshold sampling mechanism to sample the neighbor nodes with high causal correlation so that noise interference can be reduced. Considering the different influences of the neighbor nodes on the central node, we aggregate the features of the neighbor nodes according to the degree of importance. Using GAT for feature extraction can better characterize node information.
(3) An ACG-LGraphBM model is proposed, which does not require mass training samples to achieve accurate root cause localization and can provide interpretability analysis of the returned results.

This rest of paper is organized as follows. Section 2 provides an overview of the work related to root cause analysis. Section 3 introduces the preliminaries of relevant knowledge. The novel root cause analysis model is proposed in Section 4. Section 5 presents the experimental results and analysis. Section 6 summarizes this paper and envisages the future work.

2. Related work

Root cause analysis is also called root cause tracing or root cause localization. In recent years, root cause analysis has achieved extensive research in various fields such as cloud platforms (Meng et al., 2020; Qiu et al., 2020; Soldani and Brogi, 2022), telecommunication networks (He and Zhao, 2020; Jiang and Bai, 2023), and industrial production (Gaita et al., 2022; Dehbozorgi et al., 2020; Arunthavanathan et al., 2022).

Many existing works adopt data-driven methods to solve RCA problems. The common idea is to build a classifier model, such as SVM (Support Vector Machine) (Gaita et al., 2022), decision trees (Dehbozorgi et al., 2020), neural networks (Arunthavanathan et al., 2022), etc., then convert system states into features, and finally determine root causes by learning the implicit logic between features. Gaita et al. (2022) addressed the automation challenge in RCA of wafer defects by leveraging SVM classifier. Dehbozorgi et al. (2020) used a decision tree classifier to detect the root causes of faults in equipment related distribution systems. Arunthavanathan et al. (2022) integrated the SVM and neural network model to automatically diagnose the root causes of faults in complex processing systems. They captured unknown fault conditions through SVM, and trained the neural network for classifying known fault conditions and determining the root cause of the classified faults. These data-driven methods use historical data to train models and achieve accurate localization. However, they typically adopt an

end-to-end learning approach, where the input is raw data (e.g., features) and the output is directly the final target (e.g., root cause localization results). The intermediate process is unknown. This leads to a lack of interpretability in the localization results, making it difficult for operators to understand why the current localization results are obtained. In the context of cloud computing, Zhang et al. (2021) introduced the CloudRCA framework, which achieves root cause analysis by integrating monitoring metrics, log messages, and dependencies from CMDB (Configuration Management Database). With the combination of feature engineering and domain expertise, CloudRCA employs a knowledge-informed hierarchical Bayesian network (KHBN) for real-time fault root inference. While CloudRCA has shown impressive performance in practical applications, it relies significantly on extensive manual fault injection and labeled data. Ta et al. (2023) proposed a mechanism for identifying the root cause of cloud environment failures by integrating machine learning-based prediction methods. They adopt the Isolation Forest model to predict anomalous events and rank the root causes based on log scores. However, due to the Isolation Forest model is sensitivity to noisy data, this approach may be adversely affected when dealing with datasets that contain noise.

Unsupervised graph-based methods are usually used for root cause localization also. Li et al. (2022a) proposed an unsupervised root cause analysis scheme DAG-RCA, which mined the correlation of indicators from historical KPI (key performance indicator) data, constructed DAG (Directed Acyclic Graph) graphs, and used random walk to diagnose abnormal performance of radio access networks (RAN) edge nodes. Meng et al. (2020) proposed a framework named MicroCause to accurately locate the root cause monitoring indicators in microservices. MicroCause uses an improved PC (Peter-Clark) algorithm to capture the relationship of time series data to construct a fault causality graph. A novel time-cause-oriented random walk (TCORW) approach is then combined to infer the top N root causes of microservice failure. Qiu et al. (2020) used KPI data to build a causal graph of performance indicators, regarded the problem of root cause identification as a path search problem, and found the root cause indicators that lead to abnormal performance of microservices through breadth-first search (BFS). Liu et al. (2021) proposed a root cause location method named MicroHECL for the availability of microservice systems. MicroHECL dynamically constructs the service call graph of the target microservice system, then traverses the graph to analyze possible exception propagation chains, and ranks candidate root causes based on correlation analysis. Zheng et al. (2024) proposed a unified multi-modal causal structure learning method named MULAN, which was designed to identify root causes in microservice systems. It uses a log-customized language model to improve log representation and a contrastive learning method to extract modal features. To handle noise, the similarity between modalities and KPIs is measured to assess reliability and jointly learn the causal graph. Finally, random walks with restarts simulate failure propagation to identify potential root causes. Unsupervised methods do not require a large amount of historical data with root cause labels, but their implementation of root cause localization usually requires a traversal search of the graph, which results in expensive time costs.

In causal graph construction methods, PC algorithm (Kobayashi et al., 2022) and the improved algorithm based on PC algorithm (Meng et al., 2020; Qiu et al., 2020) are commonly used. PC algorithm is a constraint-based algorithm, which uses conditional independence test to learn the skeleton of the graph, and then orients the edges according to the predefined direction rules. Wu et al. (2021) used DirectLiNGAM (Direct Linear Non-Gaussian Acyclic Model) to infer causality from a non-Gaussian linear model to derive a metric causal graph that was used to infer root causes. Note that PC algorithm requires events to be independent and identically distributed, and this is not applicable in scenarios with complex topologies (e.g., cloud platforms). DirectLiNGAM has strict linear and non-Gaussian assumptions, which makes it impractical. Cai et al. (2022) proposed a topological Hawkes process (THP) to draw a connection between the graph convolution in the topology domain and the temporal convolution in time domains, so as to learn the causal structure of event sequences with topological constraints. They demonstrated the superiority of THP in both simulated and real data experiments. Wang et al. (2021) built a real-time causal graph based on the events formed by various indicators, logs, and activity summaries in the system, and calculated the root cause ranking through a custom PageRank algorithm. But the establishment of causal graphs in their approach requires a lot of domain knowledge, which costs a huge human resource overhead. Yao et al. (2024) introduced the Chain-of-Event (CoE) model for RCA in microservice systems. The model constructs an event causality graph based on the invocation relationships between microservices and learns causal link weights and event importance scores. By pruning redundant paths to eliminate false causal relationships and analyzing event propagation probabilities, the model effectively identifies the root causes.

Alarm flooding is one of the main issues in RCA, as it is difficult to identify real root cause of the failure from massive alarm logs (Jiang and Bai, 2023). Zhang et al. (2020) constructed an anomaly propagation graph based on the components that generated alarms. They tracked the anomaly propagation process and located the root cause through two optional algorithms: *random walk* and *state iteration*. Parvez et al. (2022) proposed a real-time pattern matching and ranking approach to conduct similarity analysis under an online alarm flood situation, which can predict upcoming alarm sequences and identify potential problems. Hu et al. (2023) proposed a method for identifying the root cause of industrial alarm floods based on word embedding and few-shot learning to solve the problem of alarm floods. However, Zhang et al. (2020) only focus on which components generated alarms and ignore the contents of the alarm logs, where the alarms generated by some components may not be related to the root cause. In the log-based method (Parvez et al., 2022; Hu et al., 2023), the connection relationship between the components that generate the alarm log is ignored, making it difficult to find the location of fault.

There are a few studies in combining graph-based methods with data-driven methods for RCA. On the one hand, constructing graph structures to describe the relationships between things can help to preliminarily understand the impact mechanisms between things. On the other hand, using data-driven methods for training the root cause localization model can avoid the cost of traversing the graph. Jiang and Bai (2023) proposed the Alarm Propagation Graph Neural Network (APGNN) for RCA in telecommunications networks. APGNN builds alarm propagation graphs (APG) based on frequent association mining and Bayesian network. Moreover, APGNN uses graph neural networks (GNNs) for feature extraction to learn the mapping from APG to real faults. Li et al. (2022c) proposed a fault localization approach named *DéjàVu*, which is used for recurring failures in online service systems. *DéjàVu* designs a deep learning-based model and trains it with fault dependency graphs and historical data. The model achieves good performance in a limited number of scenarios (i.e., recurring faults). Both of the aforementioned methods utilized GNNs. In the context of RCA in cloud environments, however, the challenge of data sparsity often limits the representational power of GNNs. Recently, some approaches have begun to leverage self-supervised learning to enhance the representational capabilities of GNNs. For instance, Zhao et al. (2023) proposed a self-supervised learning method that integrates and optimizes graph structure through a multi-view contrastive learning framework, thereby improving the performance of GNNs in downstream tasks. Xia et al. (2023) introduced a self-supervised learning framework capable of automatically generating data augmentation for recommendation systems without the need for manually designed contrastive views. Liu et al. (2025) designed the GCD (Graph Contrastive Denoising) module to enhance graph structure learning (GSL) on small, densely weighted EEG (Electroencephalogram) graphs through a combined pretraining and fine-tuning approach, where contrastive learning was applied by GCD during pretraining to map the original graph to an embedded representation. This can effectively filter out irrelevant information. Hu et al.

(2025) focused on training multilayer perceptron (MLP) with teacher GNNs. Their proposed framework integrated soft feature similarity into MLPs and also utilized contrastive learning to enhance the training of student MLPs. Wan, Liu, …& Wang (2024) proposed the session-based recommendation with spatiotemporal contrastive learning enhanced GNNs (RESTC).

Inspired by the excellent performance of data-driven methods in efficiency and accuracy, in this paper, we use GNNs and machine learning methods to build a root cause localization model. To describe the causal correlation between events, we use the graph-based methods to learn the causal structure of alarm events and construct an ACG. We propose an improved THP method for causal structure learning, which uses the time-delay mechanism to construct ACG and can deal with the THP's limitation that the collected alerts are always arranged in strict chronological order. For the constructed ACG, we further propose the weight threshold sampling mechanism to sample the topological neighbor nodes, ensuring that there is a strong causal correlation between the central node and the sampled neighboring nodes and reducing the interference from noise. The weight threshold sampling mechanism can filter the noise alarms based on the causal correlations in the constructed ACG. By integrating ACG, the weight threshold sampling mechanism GAT, and LGBM, the localization results include both the root cause location (i.e., the faulty component) and the root cause alarm (e.g., port 80 communication anomaly) of fault. To address the issue of lack of interpretability in data-driven methods, we additionally provide interpretability analysis of localization results.

3. Preliminaries

This section presents two basic definitions of anomaly correlation graph and abnormal event type. We also present the preliminaries of topological Hawkes process and GAT, which are used in the root cause analysis.

Definition 1. (Anomaly Correlation Graph). An ACG is a directed weighted graph expressed as $G_V=(\boldsymbol{V}, \mathbf{E}_V)$, where $\mathbf{V}$ represents the set of abnormal event types, and $\mathbf{E}_V$ represents the set of directed weighted edges. An edge $< v_i, v_j, weight_{ij} >$ exists in $\boldsymbol{E}_V$, indicating that there is a causal correlation between abnormal event type v_i and v_j, specifically, v_i is the cause of v_j, and their correlation degree is $weight_{ij}$.

Definition 2. (Abnormal Event Type). All alarm instances are classified, and alarms with similar contents are classified into the same type. All types constitute the abnormal event type set $\mathbf{V}$.

Two alarm instances a_1 "port 80 communication anomaly" and a_2 "port 8080 communication anomaly", for example, belong to the same abnormal event type, which is described as "port communication anomaly".

3.1. Topological hawkes process

Typical causal discovery methods, such as the PC algorithm and momentary conditional independence (PCMCI) test (Qu et al., 2021), usually focus on exploring the independence between causal variables and ignore the topology network behind the events. To remedy this defect, Cai et al. (2022) proposed the topological Hawkes process for learning the causal structure of topologically dependent event sequences. THP is applied to describe events, which are usually generated by nodes in the network topology. An event is not only affected by the cause event that happens in the sequence itself, but also excited or inhibited by the cause event that is in its topological neighbors. The event sequences are extended from traditional $\varepsilon = \{v_i, t_i\}$ to $\varepsilon = \{n_i, v_i, t_i\}$, where $n_i \in \boldsymbol{N}$ represents the node in the topological graph; $v_i \in \boldsymbol{V}$ represents the event type; $t_i \in \boldsymbol{T}$ represents the occurrence time of the ith event. THP is to obtain the causal structure of all event types $\boldsymbol{V}$ based on their topological structure and event sequences ε.

The intensity function of Hawkes process describes the impact of past events on event v at time t, where the intensity signifies the expected frequency of events occurring in the vicinity of time t, given all past events. THP uses graph convolution to describe the relationship between graph convolution in the topological domain and temporal convolution in the time domain. In actual business, events are usually not continuous but collected over a period of time, and attention is mainly paid to discrete time scenarios. Let Δt be the minimum time period, and $\boldsymbol{X} = \{X_{n,v,t} | n \in \boldsymbol{N}, v \in \boldsymbol{V}, t \in \boldsymbol{T}\}$ be a set of observation results, where $X_{n,v,t}$ corresponds to the observed number of occurrences of event type v at node n within time interval $[t - \Delta t, t]$. The intensity function is expressed in Eq. (1).

$$\lambda_v(n,t) = \mu_v + \sum_{v' \in \boldsymbol{PA}_v} \sum_{n' \in \boldsymbol{N}} \sum_{k=0}^{K} \alpha_{v',v,k} \hat{A}^k_{n',n} \sum_{t' \in \boldsymbol{T}_{t^-}} \kappa(t - t') X_{n',v',t'} \tag{1}$$

In Eq. (1), $\lambda_v(n, t)$ represents the cumulative effect of all causally related events at node n at time t; μ_v represents the base intensity, the fundamental frequency of events occurring in the absence of any historical event influence. We use $G_N = (\boldsymbol{N}, \boldsymbol{E}_N)$ to represent the topological structure over a set of nodes $\boldsymbol{N}$ and edges $\boldsymbol{E}_N$. Let A donate the adjacency matrix of G_N in which $A_{ij} = 1$ if $(i, j) \in \boldsymbol{E}_N$. $\hat{A} = D^{-1/2} A D^{-1/2}$ is the normalized adjacency matrix, where $D \in R^{|\boldsymbol{N}| \times |\boldsymbol{N}|}$ is the diagonal degree matrix with $D_{ii} = \sum_j A_{ij}$; $\hat{A}^k_{n',n}$ describes the subgraph of the topological graph, representing the topological relationship between n and n' from the k - hop paths; $\alpha_{v',v,k}$ represents the causal influence between $v' \to v$ from the k - hop paths, capturing the causal effect between two nodes that are indirectly connected through k nodes in the network; κ $(t - t')$ usually uses exponential form to characterize the time-decay of the causal influence; $\boldsymbol{PA}_v$ represents a set of event types for the cause of event type v; K refers to the farthest topological distance; $\boldsymbol{T}_t^- = \{t' | t' \in \boldsymbol{T}, t' < t\}$ denotes the set of all time points prior to time t.

To learn the causal structure of events, the input for THP is the topological structure G_N $(\boldsymbol{N}, \boldsymbol{E}_N)$ and event sequences, and the output is the event causality graph G_V $(\boldsymbol{V}, \boldsymbol{E}_V)$, where $\boldsymbol{E}_V$ denotes the set of edges in G_V. $\Theta = (\mu_v, \alpha_{v',v,k})_{v',\, v \in \boldsymbol{V},\, k=0,\ldots,K}$ contains the parameters of the model. The likelihood function is expressed as a function of G_V, Θ, and G_N.

$$\mathrm{L}_B(G_V, \Theta; \boldsymbol{X}, G_N) = \mathrm{Const} - \frac{p \log(m)}{2} + \sum_{v \in \boldsymbol{V}} \sum_{t \in \boldsymbol{T}} \sum_{n \in \boldsymbol{N}} \Big[- \lambda_v(n,t) \Delta t + X_{n,v,t} \log(\lambda_v(n,t)) \Big] \tag{2}$$

Bayesian penalty factor $(p\log(m)/2)$ is introduced in Eq. (2) to enhance sparsity. Enhancing sparsity in this context means increasing regularization to reduce the number of non-zero parameters, thereby mitigating the risk of overfitting. $p = |\boldsymbol{V}| + K|\boldsymbol{E}_V|$ represents the number of parameters, and m represents the total number of events that occur within time $\boldsymbol{T}$.

In the process of model training, EM (Expectation Maximization) iteration algorithm (Lewis and Mohler, 2011) is adopted to evaluate the best model parameters. The main idea of learning causal structure using THP is to start with an empty graph structure without any edges and random parameters, and then, in each iteration, an edge is added, deleted, or reversed on the current optimal graph structure so that the neighbor graph structures of the current graph structure can be obtained. Then, among these neighbor graph structures, we need to find a better graph structure than the current one until such a graph structure cannot be found. This process is aimed at identifying the optimal causal graph, which is the graph structure that best represents the causal relationships among variables and corresponds to the maximum likelihood of the observed data, capturing the most accurate and likely causal influences.

3.2. GAT

GAT is a graph neural network that introduces attention mechanism

to captures the spatial features of the topological graph. The attention layer of GAT can learn the importance of neighbor nodes to the central node according to features by assigning different coefficients to neighbor nodes and finally achieving information aggregation (Jiang et al., 2022). The input of the graph attention layer is a set of node feature vectors $h = \left\{ \vec{h}_1, \vec{h}_2, \ldots \vec{h}_N \right\}$, $\vec{h}_i \in R^F$ and topology G_N, and the output is a new set of node feature vectors $h' = \left\{ \vec{h}'_1, \vec{h}'_2, \ldots \vec{h}'_N \right\}$, $\vec{h}'_i \in R^{F'}$, where N represents the number of nodes and F/F' represents the feature dimension. Then, by learning the correlations between nodes, it updates these feature vectors to produce a new set of feature vectors. During this process, each node calculates a similarity coefficient with its neighboring nodes and itself, reflecting the strength of the relationships between nodes. Subsequently, GAT computes an attention coefficient, which is essentially a softmax operation, determining the contribution of each neighbor to the central node's feature update based on the similarity coefficient. Ultimately, each node's new feature vector is a weighted sum of its own features and its neighbors' features, with weights determined by the attention coefficients.

The introduction of multiple attentions into GAT can prevent self-attention overfitting. The multi-head attention mechanism (Zhang et al., 2021) can make the model more stable. Fig. 1 shows a schematic diagram of $node_1$'s multi-head attention (with three heads) on its neighborhood. Different arrow colors represent independent attention calculations, and aggregate features from each attentional head are concatenated or averaged in order to obtain $\vec{h}'_1$. If the aggregated features of each attention head are concatenated, Eq. (3) specifically describes this aggregation process, resulting in the aggregated feature $\vec{h}'_i$ for node i. In Eq. (3), $\vec{h}'_i$ represents the new features of the node i, which integrate the domain information; $\sigma(\bullet)$ is the activation function; α_{ij} denotes the attention coefficient between node i and its neighbor j ($j \in \mathcal{N}_i$); W represents the learnable parameter; K indicates the number of attention heads; $\|$ represents the concatenate operation. The multi-head attention mechanism can play an integrated role, helping the network to capture richer information. Therefore, we use the multi-head attention mechanism of GAT to extract text features of nodes. These features are numerical representations derived from alarm data through an encoding process, which translates the raw textual information of alarm messages into a form that is amenable to model processing and subsequent analysis.

$$\vec{h}'_i = \Big\|_{k=1}^{K} \sigma\left(\sum_{j \in \mathcal{N}_i} \alpha_{ij}^k W^k \vec{h}_j \right) \tag{3}$$

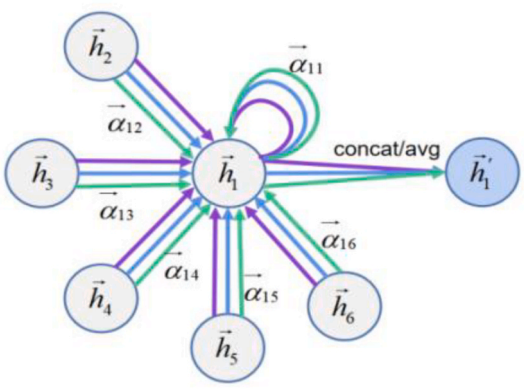


Fig. 1. Schematic diagram of GAT multi-head attention mechanism.

4. Root cause analysis based on ACG-LGraphBM

The goal of RCA is to identify the root cause node and root cause alarm when a fault occurs (e.g., a root cause node: $node_{22}$; a root cause alarm: the application stops running). Their combination can inform engineers the fault location and event simultaneously, providing operability for the fault (Li et al., 2022c). When a fault occurs, the numbers of different alarms generated at the root cause node very varies and the occurrence times of root cause alarms are much greater than that of non-root cause alarms. So, to simplify the problem, we mainly concentrate on locating the root cause node. On this basis, statistical methods are used to find the root cause alarm.

4.1. Overview

We propose the fault root cause analysis model ACG-LGraphBM in this paper, which achieves effective node feature extraction and accurate root cause localization. Its processing framework of root cause analysis is shown in Fig. 2. First, the collected alarm data are processed by classification and vectorized representation. The processed data are used as the input together with the topology. With the processed historical data, the causal correlations of abnormal alarm events are then learned to construct the anomaly correlation graph. After that, node sampling is realized by the weight threshold sampling mechanism, and node feature extraction and classification are respectively realized by GAT and LGBM. Finally, new alarms appear can be input into the trained model for diagnostic analysis, and root cause localization results and interpretable reports are obtained for the engineers' reference.

The architecture of ACG-LGraphBM model is shown in Fig. 3, including data processing as input, ACG layer, layer GAT, LGBM layer, and output layer. The model takes alarm logs and topological structure as inputs. The original alarm logs are preprocessed to obtain two types of node features: *text features* and *statistic features*. The statistic features include node ID, system ID, number of alarms, and number of the root causes in history. The initial text features are obtained by text classification, one-hot encoding, and mean vector. For the text features, we classify and vectorize the alarm log texts. First, we use the method of Jieba word segmentation to segment the log data and then use TF-IDF (Term Frequency-Inverse Document Frequency) to represent text features. Then we use TF-IDF to represent text features. Based on text features and type labeling, the Multinomial Naïve Bayes model is trained, which is a commonly used model for text classification. The vectorization of log is obtained by one-hot coding the classification results. The data processing of the ACG-LGraphBM model is detailed in Section 5.1.1.

Three core modules ACG, GAT, and LGBM in ACG-LGraphBM have distinct roles. The ACG layer is applied for anomaly correlation graph construction and node sampling, the GAT layer focuses on node feature extraction, and the LGBM layer is responsible for classification. The Time-delay Topology Hawkes Process (TTHP) is employed to learn the causal structure of topologically related alarms (alarms also referred to as abnormal events), enabling the construction of ACG. The structure of ACG is represented by an adjacency matrix of size $N_V * N_V$, where N_V represents the number of abnormal event types. To quantify the degree of correlation between alarms in ACG, conditional probability method is utilized to calculate the weights of the causal edges.

When a node in the topology network experiences a fault and triggers an alarm, the connected nodes also generate abnormal alarms. So, it is necessary to extract the text features of node alarm logs while considering the spatial information. Simple statistical methods often cannot fully capture such complex dependencies and features. The GAT layer is hereby applied to extract node text features. With its attention mechanism that can assign different attention coefficients to neighboring nodes, it takes the topological graph data and the N_V-dimensional initial vector of alarm logs as input, and outputs the 64-dimensional vector that represents the log text features of nodes by aggregating neighbor node

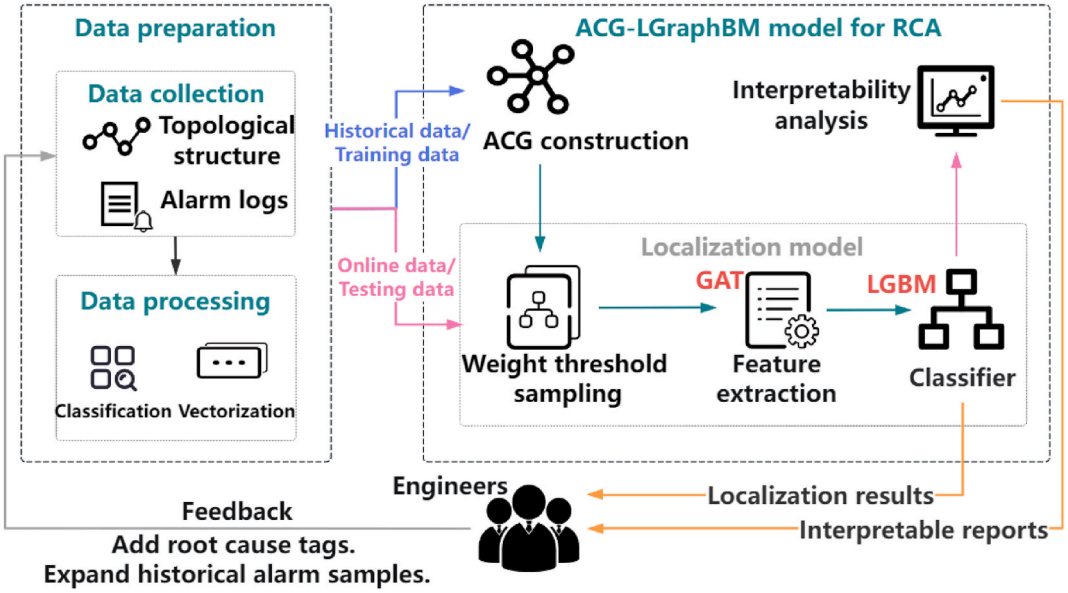


Fig. 2. Processing framework of root cause analysis based on ACG-LGraphBM.

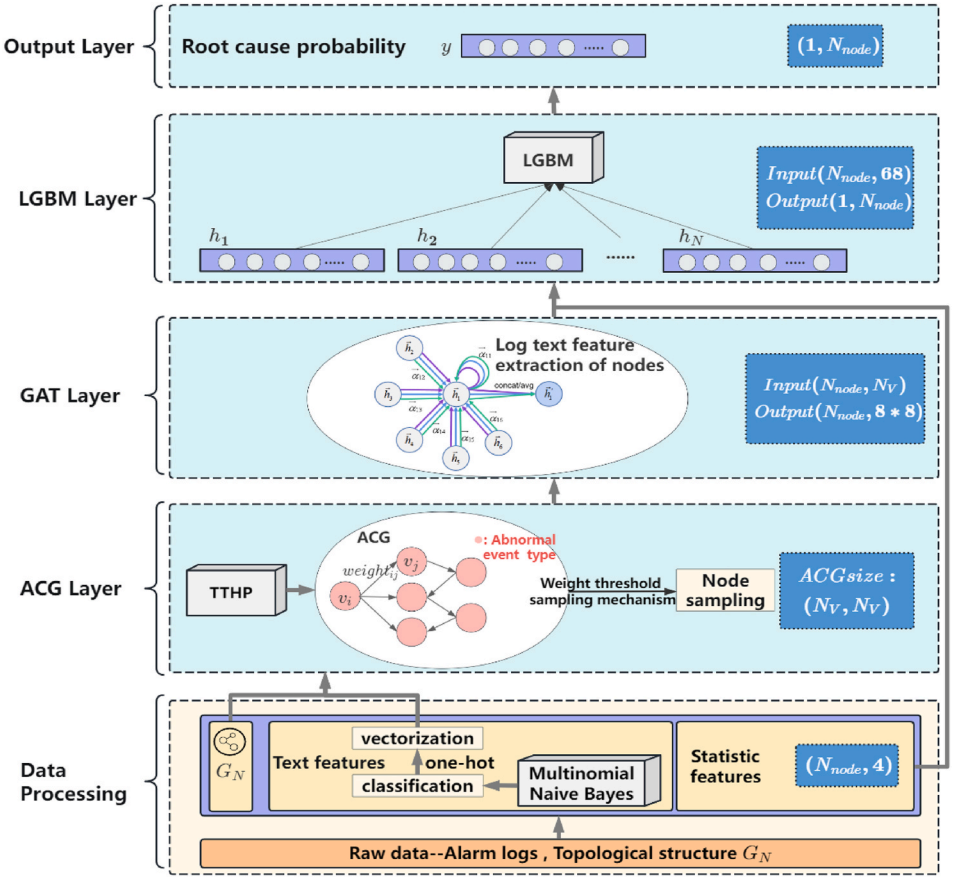


Fig. 3. Architecture of ACG-LGraphBM model that contains three core layers of ACG, GAT and LGBM.

features for each node. Note that, however, alarm logs often contain noise and GAT cannot make further judgments on alarm information. To prevent noise being incorporated into the extracted node features, we propose a weight threshold sampling mechanism to sample the neighboring nodes which correlation degrees are not lower than the threshold, based on the weights of ACG edges and the alarm information generated by adjacent nodes. This can help reduce the interference caused by noise alarms.

While GAT achieves favorable results in node feature extraction, training stable deep neural network models requires a substantial number of training samples and significant computational resources (Deng et al., 2021). The LGBM layer is applied to further analyze the

features and train a classifier, aiming to construct a precise fault root cause localization model. It concatenates the log text features and statistic features of nodes, and classifies them according to the features. Its input is 68-dimensional feature data and the output is the probability that each node is the root cause.

For the identified root cause of fault with ACG-LGraphBM, we further propose a similarity-based interpretability analysis method to explain why the current localization result is obtained, which helps engineers trust the localization result. The interpretability analysis process is shown in Fig. 4. The basic idea is that the same fault results should indicate their similar features. Therefore, the features extracted from the root cause node at the GAT layer are compared pairwise with the features of historical fault set's root cause nodes, where The Euclidean distance (You et al., 2022) is used as the metric. Then the top k fault sets with the highest similarity are identified, suggesting that the model learns from these similar historical failures to locate the root cause. Finally, the interpretability report is sent to the engineer together with the root cause localization result. Engineers can directly refer to the maintenance work orders of the similar historical faults to repair the current fault.

4.2. Anomaly correlation graph construction

We construct an anomaly correlation graph to represent the causal correlation and degree of correlation between alarms. The construction consists of two main parts: causal structure learning and causal edge weight calculation. For the causal structure learning, unlike many existing studies (Zhang et al., 2020; Jiang and Bai, 2023) that only emphasize alarm correlation, we highlight the causation between alarms. Correlation only indicates the existence of correlations between events and cannot demonstrate the relationship between cause and effect, while causation involves deeper relationships where one event can directly or indirectly lead to another. For example, correlation can be manifested as the simultaneous occurrence of alarms a_x and a_y, while causation can be manifested as the occurrence of a_x leading to the occurrence of a_y. The analysis of alarm causality can more intuitively reveal the causal effects between alarms generated by topology connected nodes, which helps to accurately locate the root cause. In cloud environments, most alarm events are generated by nodes in the topology network. Therefore, when learning the causality of alarm events, it is necessary to consider not only the impact of the alarm event sequences generated by the node itself, but also the impact of the topological neighbors of the node.

THP (Cai et al., 2022) has made a breakthrough in the causal structure learning of topological dependent event sequences. However, THP assumes that the collected alarms are always in strict chronological order, ignoring time errors when the influence between events is considered. In actual production environments, there may be delays in the collection of some alarms due to the influence of collection intervals and collection errors. Usually, alarm logs are not obtained strictly in the chronological order of the occurrence of anomalies. Considering the impact of this part of error, we add the processing of time-delay to THP in order to more comprehensively aggregate the impact between alarm events in real operation and maintenance data. Then we have Time-delay Topological Hawkes Process (TTHP), a variant of THP, for causal structure learning.

In TTHP, an alarm event is affected not only by the alarm events that appear earlier than the given alarm event in the time record, but also by some alarm events that are collected at the same time as or later than the occurrence of the given alarm event within the delay time interval. Then the intensity function of Eq. (1) is changed to:

$$\lambda_v(n,t) = \mu_v + \sum_{v' \in \boldsymbol{PA}_v} \sum_{n' \in \boldsymbol{N}} \sum_{k=0}^{K} \alpha_{v',v,k} \widehat{A}^k_{n',n} \sum_{t' \in \boldsymbol{T}_{t^-_{,t}+\Delta t_l}} \kappa(|t-t'|) X_{n',v',t'} \tag{4}$$

where Δt_l represents the delay time interval. Then, the likelihood value (L_B) of the Hawkes process is calculated by Eq. (2). The graph structure that corresponds to the maximum likelihood is obtained as the final causal structure G_V* (Cai et al., 2022). G_V* is a directed unweighted graph, where each edge is referred to as a causal correlation edge.

In addition to causality, the correlation degree between abnormal events is also important information. The causal relationships learned through TTHP may not be entirely correct. Therefore, it is necessary to further ascertain the validity of these causal relationships through the degree of correlation. For example, we learn a causal relationship of $v_4 \rightarrow v_{18}$, which means that "port communication anomaly" is the cause of "SDD IO usage lasts n minutes greater than the threshold". However, it can be observed that the occurrence of v_4-type events rarely leads to the occurrence of v_{18}-type events. That is, the causal relationship between them is very weak, which is likely learned incorrectly. Clearly, it is necessary to calculate the degree of correlation between abnormal events in order to filter weak causal relationships.

For the causal edge weight calculation in the ACG construction, with the obtained causal correlation edges of the abnormal event types, we can calculate the weights of edges to represent the degree of correlation between abnormal alarm events. First, we slice the historical data based on time. For abnormal alarm event data in each group of time slices, we construct abnormal event statistics samples. The occurrence of abnormal event types within the time slice is marked as 1, otherwise it is 0. An example of abnormal event statistics sample construction is shown in Table 1. According to the obtained abnormal event statistics samples and causal correlation edges, we next use conditional probability to

Table 1
Example of abnormal event statistics sample construction.

Time slice	v_0	v_1	v_2	v_3	v_4
Time slice-1	1	1	0	0	1
Time slice-2	1	1	1	0	0
Time slice-3	1	0	1	1	1

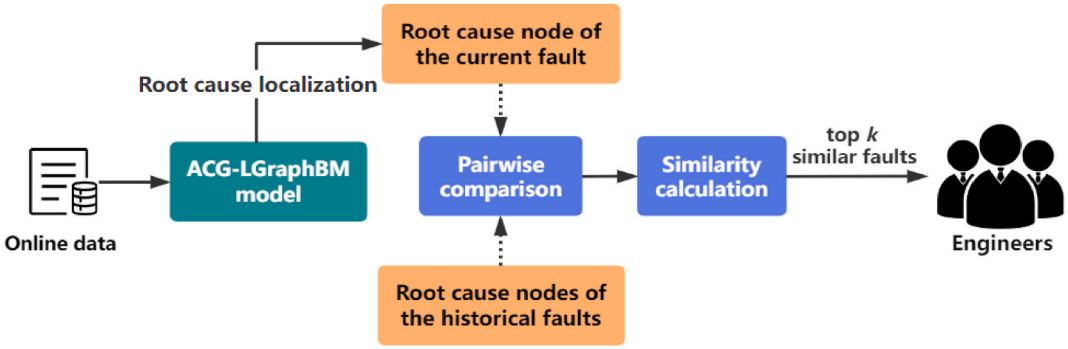


Fig. 4. Interpretability analysis process of the identified fault root cause.

calculate the weight of edge. All abnormal event types are regarded as causal nodes, and the weight formula for the causal correlation edge (say, $v_i \rightarrow v_j$) can be expressed by Eq. (5). The ratio of the occurrence number of the result nodes under the condition that the cause node occurs to the total number of occurrences of the cause node is used as the weight of the causal correlation edge.

$$weight_{ij} = \frac{count_{v_j/v_i}}{count_{v_i}} \tag{5}$$

Algorithm 1 provides a detailed process of constructing ACG. The input is the collected alarm data X, which contains information about the alarm event sequences and the topology network behind the alarms. In the causal structure learning part (lines 1-10 of Algorithm 1), the starting point is an empty graph structure without any connected edges. In each iteration, the neighbor graph set $Nei(G_V^*)$ of the current optimal graph G_V^* is generated. Each neighbor graph $G_V' \in Nei(G_V^*)$ is an edge operation on the current optimal graph structure, including: adding, deleting, and reversing. We want to find the neighbor graph with the highest likelihood value L_B', which is the current optimal graph and the starting point for the next iteration. We do successive iterations until the likelihood value no longer increases, and the current graph structure G_V^* is the final causal graph structure. Next, we add the nodes and edges in G_V^* to G_V, and calculate the weight of each causal correlation edge in G_V^* using the conditional probability method (lines 12-17 of Algorithm 1). We ultimately obtain an anomaly correlation graph G_V, which contains causal correlation information and weight information.

Algorithm 1. Anomaly correlation graph construction

Input: The collected alarm data X
Output: G_V

1: $G_V^{\sim} \leftarrow$ empty graph, $L_B^{\sim} \leftarrow -\infty$, $L_B^* \leftarrow L_B^{\sim} - 1$
2: initialize the model parameters $\Theta^{\sim}$ randomly
3: **while** $L_B^* < L_B^{\sim}$ **do**
4: $\langle G_V^*, \Theta^*, L_B^* \rangle \leftarrow \langle G_V^{\sim}, \Theta^{\sim}, L_B^{\sim} \rangle$
5: **for** every $G_V' \in Nei(G_V^*)$ **do**
6: initialize Θ' randomly
7: repeat update L_B' and Θ' until convergence
8: **end for**
9: $\langle G_V^{\sim}, \Theta^{\sim}, L_B^{\sim} \rangle \leftarrow \langle G_V', \Theta', L_B' \rangle$ with largest L_B'
10: **end while**
11: add all nodes in G_V^* to G_V
12: **for** every $e \in$ (edges in G_V^*) **do**
13: $v_i \leftarrow$ e.source
14: $v_j \leftarrow$ e.destination
15: calculate $weight_{ij}$ by Eq. (5)
16: add edge $< v_i, v_j, weight_{ij} >$ to G_V
17: **end for**
18: **return** G_V

Fig. 5 shows a partial anomaly correlation graph constructed with Algorithm 1, where each node represents a type of abnormal event, the edge represents the causal correlation between nodes, and the weight of edge represents the degree of causal correlation. An edge $v_4 \overset{0.92}{\rightarrow} v_0$ in Fig. 5, for example, can be explained as that the abnormal event type v_4 is the cause of v_0. That is, "port communication anomaly" may lead to "the request delay exceeds the threshold", and the correlation degree between the two types of alarm events is 0.92. Note that the input of Algorithm 1 needs a data processing for raw data about alarm logs and topological structure, which is depict ed in Section 5.1.1.

4.3. Weight threshold sampling

For each node in the topology network of cloud platform, the GAT algorithm aggregates the features of all its neighbor nodes with the central node, ensuring that the extracted node features contain spatial information. However, in O&M scenarios, alarms generated by two topologically connected nodes are not necessarily related. Let us look at an example. Let an alarm of type v_4 be a root cause alarm. Then, it will cause the alarms of type v_0, v_1, v_7, and etc. in its neighborhood. If node R generates an alarm of type v_4 (e.g., "port 80 communication anomaly") and other nodes connected to node R generate alarms of type v_0, v_1, and v_7, node R is highly likely to be the root cause node. Let node B be a node connected topologically to node R. Node B generates an alarm of type v_{14} (e.g., "The average received traffic of network card eth0 exceeds 180 MB for 10 min"). Clearly the alarm generated by node B is not caused by node R, but this noise information will be aggregated into the features of node R during feature extraction, which may affect the localization of the root cause. Noise refers to the alarms that are not causally related or weakly causally related to the alarms generated by the parent node. To deal with this issue, we propose a weight threshold sampling mechanism, which can selectively sample neighbor nodes during the node feature extraction process. That is, the nodes whose correlation degree between alarms is greater than the given threshold are sampled so that the interference of noise alarms in O&M scenarios can be reduced.

The alarm information collected during a period of time is first converged to the nodes. Each node with alarm information is represented as $node_i$ - v_x, which means that $node_i$ generates an abnormal alarm event type v_x. A node may have multiple generated alarms. For the convenience of observing the degree of causal correlation between nodes, v_x is the alarm event type with the most occurrences of the node. Then, the correlation degree between alarm events is obtained from the ACG, which is the initial weight of the topological edges. Fig. 6 shows the direction of causes and depends-on relationships between nodes. It can be observed that the fault propagation direction and the invocation direction are opposite. Assume that there are two adjacent nodes in the topology and they generate the alarms: $node_i$ - v_x and $node_j$ - v_y, in where the topology direction is $node_i \rightarrow node_j$. Then the topological edge weight of nodes i and j is the weight of the edge $v_y \rightarrow v_x$ (i.e., $weight_{yx}$) in the ACG. If there is not an edge $v_y \rightarrow v_x$ in the ACG, the weight is 0.

We set a weight threshold. Then, for the neighbor nodes whose topological edge weight with the central node is not less than the threshold, they are sampled as aggregation targets and the features are aggregated in the feature extraction process. If the weights are 0 or less than the threshold, the alarms generated by the nodes have a weak causal correlation, which are noise alarms with a high probability. At this point, these neighbor nodes can be filtered.

Fig. 7 is an example of the weight threshold sampling. For the central node 36, the alarms generated by its neighbors $node_{35}$ and $node_1$ are not caused by this node. The alarms generated by $node_{35}$ and $node_1$ are actually noise alarms. We sample these two neighbor nodes by setting a weight threshold (e.g., $weight_{threshold} = 0.8$). While extracting the features of the center $node_{36}$, the neighbors $node_{35}$ and $node_1$ are filtered because both $weight_{(36,35)}$ and $weight_{(36,1)}$ are less than $weight_{threshold}$, and only $node_{52}$, $node_{71}$, $node_{40}$ and $node_{46}$ can be sampled. The noise interference is thereby reduced.

Node sampling can essentially be viewed as a process of filtering nodes and edges in a topological structure. For each set of alarm samples, an alarm correlation subgraph (ACSG) is separately constructed, in which the edges represent strong causal correlation between the alarms generated by adjacent nodes. Fig. 8 shows an example of ACSG construction. For the original topology, we first preserve the nodes with alarms and the topological edges between these nodes so that the topological subgraph can be obtained. Then, based on the weight threshold sampling mechanism, some topological edges in the topology subgraph will be filtered. If the correlation degree between alarms generated by adjacent nodes in the topology is not less than the threshold, the edges between nodes are retained. In this way, for each group of alarm samples, the alarm correlation subgraph G_S is obtained after removing some topological edges.

4.4. Feature extraction

In the cloud platform scenario, when a fault occurs, adjacent nodes often trigger anomalies and generate alarm information due to the

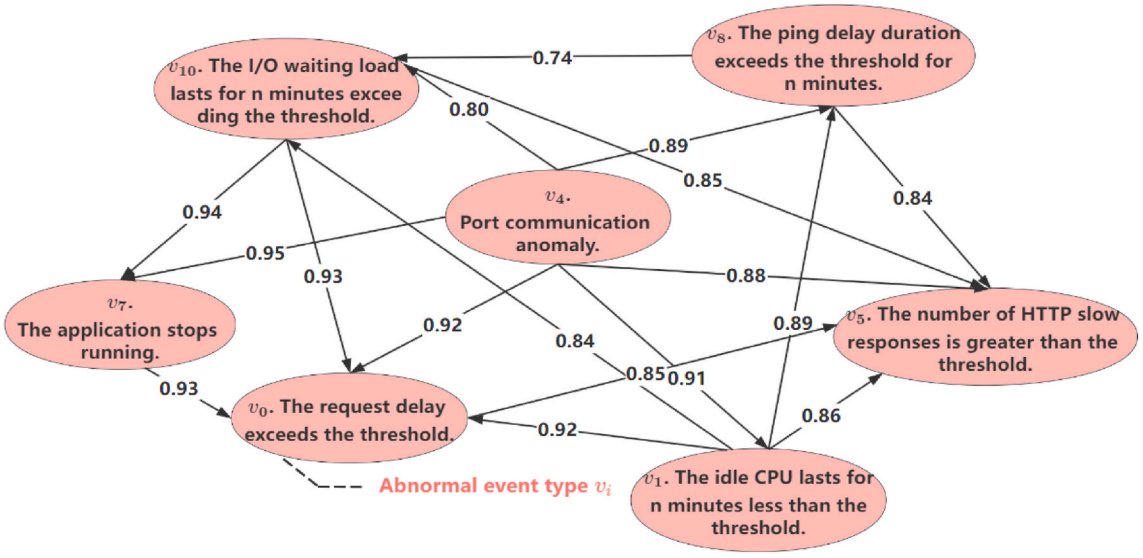


Fig. 5. An example of the constructed anomaly correlation graph with weights.

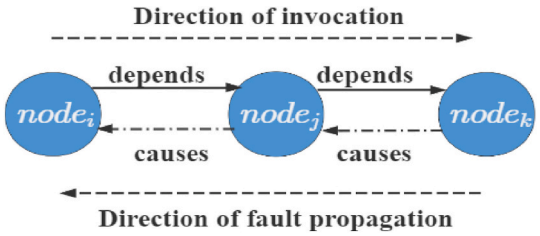


Fig. 6. Two opposite directions of fault propagation (cause relationship) and invocation (depends-on relationship).

influence of the root cause node. Therefore, in the feature extraction module, the graph embedding representation learning method is used to extract the features of nodes and aggregate their neighborhood information. The graph structure processed by Graph Convolutional Network (GCN) (Zhang et al., 2019) is not scalable and not applicable in dynamically changing cloud platforms. Due to the different relationships between any pair of nodes, it is not suitable to use GraphSage (Hamilton et al., 2017) to simply aggregate the features of the connected nodes by using the average value. To learn the representation of each node in the topological structure, we use GAT to achieve weighted aggregation of neighbor nodes (Jiang et al., 2022). The GAT model is suitable for inductive learning problems because it can achieve inductive aggregation of node features through the masked attention mechanism, avoiding the need to retrain the embedded features of nodes every time. In addition, we also use the GAT multi-head attention to improve the capacity and stabilize the training process. With the GAT multi-head attention, more robust feature mapping rules can be learned

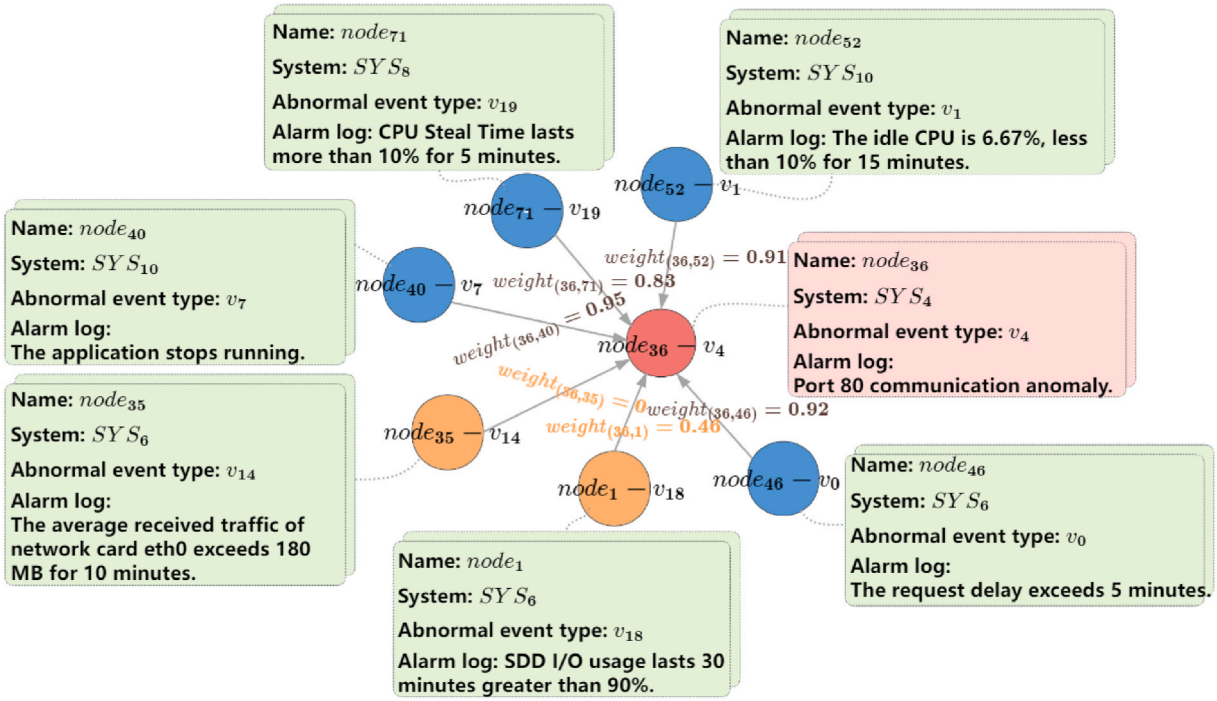


Fig. 7. An example of the weight threshold sampling for the constructed anomaly correlation graph.

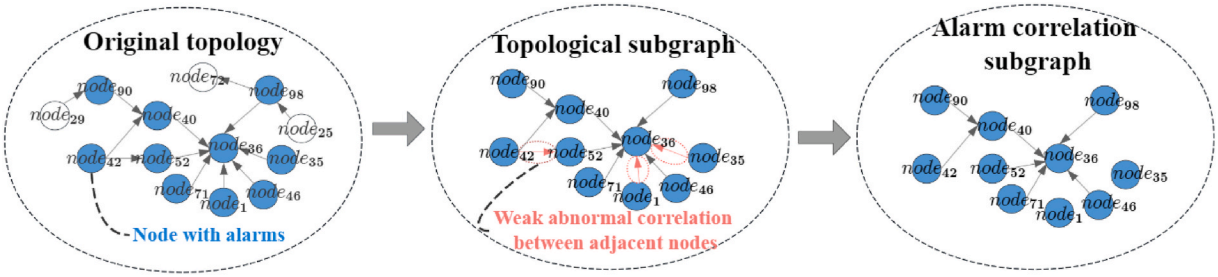


Fig. 8. An example of how to construct an alarm correlation subgraph from the original topology.

to ensure more stable feature extraction for new graphs or new nodes.

The input of the GAT layer is the initial log text features of nodes and the alarm correlation subgraph G_S. After the features are aggregated, the log text features of nodes are obtained (the implementation details of GAT are described in Section 3.2). In Eq. (6), $h^{initLog}_{node_i}$ represents the initial log text features of each node, $\|$ represents the concatenate operation. We set the number of heads for multi-head attention as 8 and the number of hidden units as 8. Then 64-dimensional text features of nodes can be obtained after feature extraction with GAT.

$$h^{log}_{node_i} = \Big\|_{k=1}^{K} \sigma\left(\sum_{node_j \in \mathcal{F}_{node_i}} \alpha^{k}_{node_i node_j} W^{k} h^{initLog}_{node_j}\right) \tag{6}$$

4.5. Classifier

Deep learning-based root cause localization requires a large number of training samples and dense computing power (Deng et al., 2021). In real application, root cause alarms usually account for a small proportion of alarm data only (Jiang and Bai, 2023) and the number of labeled training samples is relatively small. LGBM is an efficient implementation of the GBDT (Gradient Boosting Decision Tree) framework. Compared to traditional GBDT models, LGBM is more suitable for processing massive industrial grade data (Ke et al., 2017). In the ACG-LGraphBM model, we use LGBM classifier to achieve satisfactory performance, which assigns a suspicious score in [0, 1] to each node.

The alarm data contains statistic features and can provide information for root cause analysis. The log text features extracted from GAT are concatenated with statistic features, and the node features after concatenation are used as the input of LGBM. We train the classifier according to the input node features and root cause labeling. In order to facilitate the observation of the root cause ranking results, the classifier outputs the probability that each node is the root cause. Then we can obtain the nodes that are most likely root causes according to the ordering sequence. Note that some alarm samples do not have root causes. Therefore, it is necessary to set a probability threshold $RCA_{threshold}$, determining whether a node is a root cause. As shown in Eq. (7), when the predicted probability is not less than the threshold, the node is considered as the root cause; otherwise, the node is considered not the root cause.

$$Y = \begin{cases} 1 & \text{predict} \geq RCA_{threshold} \\ 0 & \text{predict} < RCA_{threshold} \end{cases} \tag{7}$$

The objective function during training is the F1 score. The model parameter tuning is performed with Grid Search (Sun et al., 2021). This tuning method tries every possibility by looping through all candidate parameter selections, and the best performing parameter is the final choice. After the training based on historical data, the RCA model M^{RCA} is obtained. Then, the online alarm data can be directly input into the model for root cause localization. Algorithm 2 describes the pseudocode for root cause localization using the ACG-LGraphBM model.

Algorithm 2. Root cause localization by ACG-LGraphBM model (M^{RCA})

Input: AlarmSample, G_N, M^{RCA}
Output: RootNodeList
1: NodeSample = AlarmSample.groupby ('node')
2: **for** item in NodeSample **do**
3: $type^{log}$ = TextClassifier (item ['log'])
4: V^{log} = onehot ($type^{log}$)
5: generate $h^{statistic}$ from item, $h^{initLog}$ = MeanVector (V^{log})
6: result = M^{RCA} ($h^{statistic}$, $h^{initLog}$, G_N)
7: **if** result ≥ $RCA_{threshold}$ **then**
8: add result to resultList
9: **end if**
10: **end for**
11: sort (resultList)
12: RootNodeList = getNodeTopK (resultList)
13: **return** RootNodeList

In the process of root cause localization based on the topological structure and the alarm sample data obtained during the monitoring period, the sample data is first grouped according to different nodes (line 1). The log text is classified by the trained text classifier (line 3), and then the log vector is represented by the one-hot encoding of the class (line 4). The initial vector representation of a node includes two parts: the statistical characteristics from the sample and the log text vector of node (line 5). The log text vector of a node is represented by the mean vector of all alarm log vectors generated by the node. Next, the topology and the initial node features are fed into the ACG-LGraphBM model, and the result is the probability that the node as the root cause (line 6). We set a probability threshold to determine whether the node is a root cause (lines 7-9). After that, the root cause probability results are sorted from largest to smallest (line 11). Finally, the top K nodes with the highest probability from the root cause candidate list are the root cause nodes (line 12).

4.6. Case study

We demonstrate how to perform RCA using ACG-LGraphBM through a case study from Suning Dataset (see Section 5.1.1 for details). As shown in Fig. 9, after obtaining the topological structure and preprocessed alarm log data, the model first performs node sampling and constructs an alarm correlation subgraph based on the current alarm data. Then, based on the subgraph, node feature extraction and classification are carried out to achieve root cause localization. In the output results, the first result obtained is a sorted list of root cause nodes, where the higher the ranking, the more likely the node is to be a root cause node. Since root cause alarms are generated by root cause nodes, the alarms generated by nodes with top 1 are arranged in reverse order of frequency, where the one with the highest number of times is the root cause alarm. At this point, the information of the root cause location and event is obtained. In this case, the root cause node is $node_{36}$, which represents the starting location of the fault occurrence. The root cause alarm is "*port 80 communication anomaly*", which indicates the event that caused the fault.

Since the model is constructed based on deep learning that is essentially a black-box model, it is difficult for engineers to derive

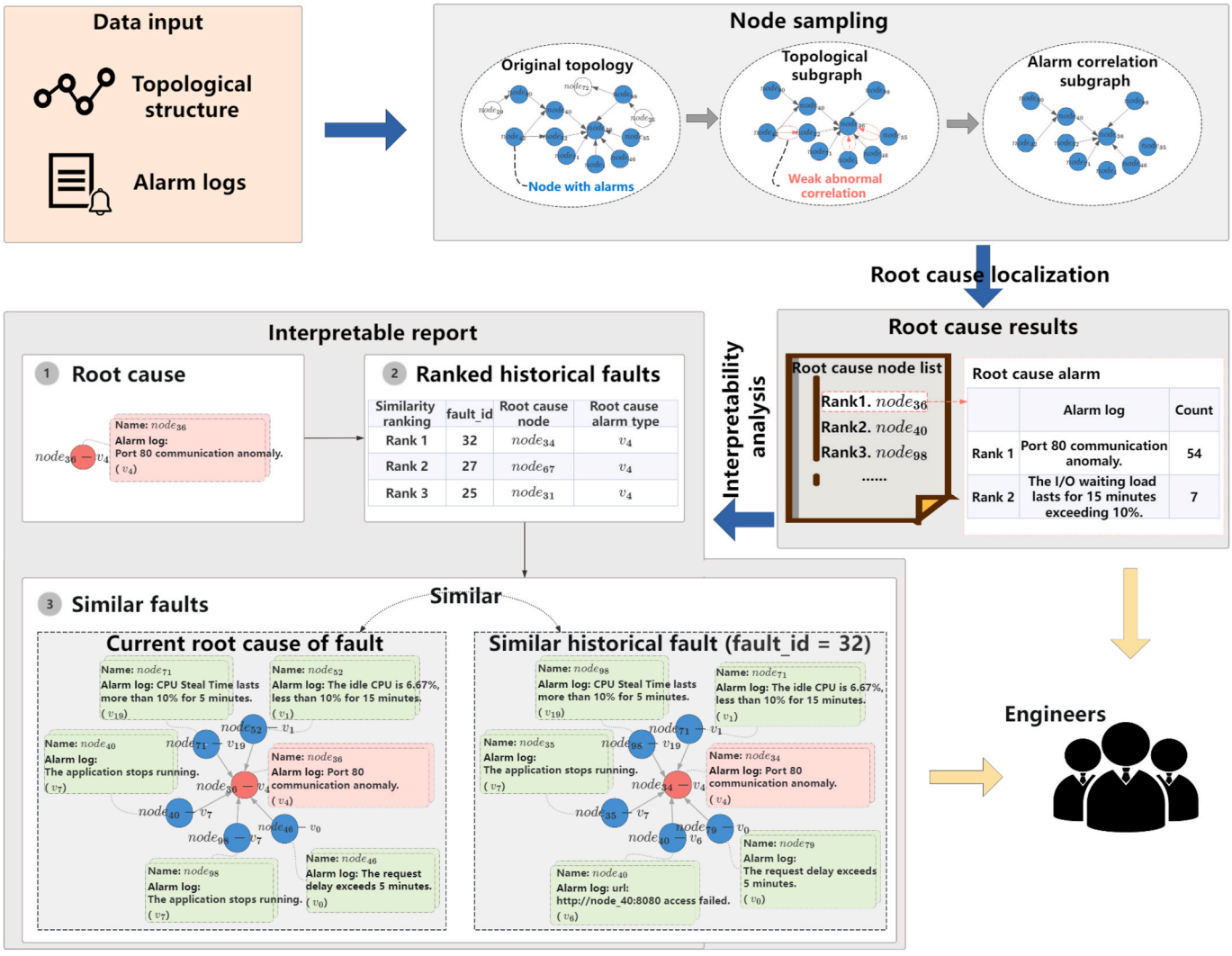


Fig. 9. A case of root cause analysis by ACG-LGraphBM.

interpretable insights from it. Our proposed model can further perform interpretability analysis by analyzing the similarities between the identified root cause results and the historical faults, and finally obtaining the most similar historical faults. Let us look at the example shown in Fig. 9. With the interpretability analysis module, the features of the identified root cause node "$node_{36}$", which abnormal even type is "$v4$" and alarm log is "*port 80 communication anomaly*", are compared pairwise with the features of nodes in the historical fault set. By ranking the similarities, the historical fault with "$fault_id = 32$", which root cause node is "$node_{34}$", has the highest similarity with the identified root cause fault in the ranked historical faults. These two root cause faults have the same abnormal even type "$v4$", which causes the similar alarm events of adjacent nodes (i.e., the alarms with type v_1, v_{19}, v_7 and v_0). The similar historical fault with $fault_id = 32$ in the interpretable report sent to engineers can help them to learn more information about the localized root cause node and root cause alarm. Furthermore, engineers can take appropriate measures to repair the current fault by referring to the maintenance work orders of the similar historical faults.

5. Experimental results and analysis

This section verifies the effectiveness of the proposed method. We first verify the function of the anomaly correlation graph construction method and then evaluate the performance of the ACG-LGraphBM model. We also conduct ablation experiments to verify the validity of each module in the proposed model.

5.1. Experimental setup

5.1.1. Dataset description and processing

We use two datasets for different experiments. The first dataset, Suning Dataset,[2] is an alarm dataset from the e-commerce platform provided by Suning, which is collected from actual O&M scenarios. We use two types of data in the Suning Dataset: *alarm* log *data* and *network topology data*. Among them, the alarm log data contains 100 sets of alarm data with root cause tags in different time periods. Each set of alarm data ranges from 10 to 15 min and 522 to 1543 alarm events, totaling 84,304 alarm events. The network topology data contains the topological connection relationship between nodes, with a total of 100 nodes. Each node represents a component in the e-commerce platform, and each component belongs to a system. There are a total of 10 systems. The names of the systems and components are desensitized, which are respectively represented by SYS_1 - SYS_{10} and $node_0$ - $node_{99}$. The Suning Dataset is described in Table 2.

The second dataset, Wireless Dataset,[3] is a wireless network dataset made public by Huawei after desensitizing the actual device alarm data

[2] Suning Dataset: http://www.cnsoftbei.com/plus/view.php?aid=479.
[3] Wireless Dataset: https://github.com/gcastle-hub/dataset.

Table 2
Description of the suning dataset.

Alarm log data				Network topology data	
Number of sets	The time range of each set	Number of alarm events in each set	Total number of alarm events	Number of nodes	Number of systems
100	10 - 15 min	522 - 1543	84,304	100	10

and hiding the device name and alarm name. This dataset contains 34,839 alarm event records, including 18 event types and 55 devices of topological network nodes. The causal relationships behind the data are given by domain experts. Examples of Wireless Dataset are given in Table 3, where each record contains alarm id, device id and timestamp information. It is shown in Table 3 that both alarms and devices are mapped into id numbers, which can be used directly for experiments.

In the data preparation stage, two types of node features are obtained from the Suning Dataset: *text features* and *statistic features*. The flow of data processing is shown in Fig. 10. We first classify and vectorize the alarm log texts, and then use the mean vector to represent the initial text features of nodes. Statistic features are statistically obtained from the alarm log data (mainly the number and quantity information). Generally, logs are processed with template matching (Ni et al., 2019), in which the templates are first extracted and labeled with types, and then logs are classified according to the templates by using methods of regular matching. This method can be well adapted to the logs that are automatically generated by the system. Note that it is possible that alarm logs may be manually recorded by engineers in O&M scenarios, and template matching cannot take effect at this point. Therefore, we use a supervised approach to train log data and build a text classifier so that new alarms can also be classified according to pre-set categories. To facilitate the labeling process, we employ the Kmeans clustering method (Ghazal et al., 2021) on sentence-level IF-IDF encoded vectors (Qaiser and Ali, 2018). The number of clusters is determined through manual adjustment. We first determine a rough range and then adjust to achieve the best clustering effect.

In the process of text classification, to perform text preprocessing, we first use the method of Jieba word segmentation (Ding et al., 2021) to segment the log data. Jieba uses a rule-based method to intercept sentences through dictionaries. Then we use TF-IDF to represent text features, which is a statistical method used to evaluate the importance of a word in a corpus for one of the files. After that, the Multinomial Naïve Bayes model (Marques-Silva et al., 2020) is trained based on text features and type labeling. The Multinomial Naïve Bayes model is a commonly used model for text classification, where word frequency is used as the main feature to establish a probability model between classes. In addition, we evaluate the accuracy of classification based on pre-manually labeled training and validation sets to ensure the effectiveness of the classification method. The vectorization representation of log is obtained by one-hot coding the classification results. The initial text features of a node are represented by the mean vector of all alarm

Table 3
Examples of the wireless dataset.

alarm_id	device_id	start_timestamp	end_timestamp
11	28	30685	30835
1	28	30685	32414

log vectors generated by the node during the current collection time period. At this point, the initial text features and statistic features of the nodes, as well as the abnormal event types, can be obtained.

5.1.2. Evaluation indexes

The main evaluation indexes used in our experiment include *Recall*, *Precision*, *Accuracy*, *F1*, and Pre@*K*. *Recall* refers to the proportion of groups with root causes that are successfully identified by the model out of all groups that actually contain root causes. *Precision* measures the proportion of the model's identified root causes that are correct. *Accuracy* is the proportion of groups that the model correctly determines to have or not have root causes out of the total number of groups. The *F1* Score, as the harmonic mean of Precision and Recall, provides a comprehensive metric for measuring the model's accuracy.

The Pre@K index is used to calculate the probability that the model can accurately locate the root cause node when it locates the top K nodes that are most likely to be root causes for each failure sample. Specifically, the definition of Pre@K, as shown in Eq. (8), measures the model's ability to correctly identify the root cause node among the top K nodes predicted. In Eq. (8), M represents all fault sets; $Root^m$ represents the actual root cause list set of fault sample m; V^m represents the root cause list set obtained by RCA of fault sample m (sorted by probability from maximum to minimum). It is easy to observe that when the K value is smaller and the value of the Pre@K is higher, the model can achieve better results in RCA.

$$\text{Pre@}K = \frac{1}{|M|}\sum_{m\in M}\frac{\sum_{i<K} V^m[i] \in Root^m}{\min(K, |Root^m|)} \tag{8}$$

5.2. Evaluation of anomaly correlation graph construction

The computer environment of experiment is the Win10 operating system, and the processor specifications are AMD Ryzen 7 5800H with Radeon Graphics of 3.20 GHz and memory of 16.0 GB. The code environment is Python 3.7. In the experiment, the training and testing sets are divided in a ratio of 7:3.

The goal of constructing ACG is to learn the causal correlation between alarm events, which can assist in feature extraction and improve the precision of root cause localization. The effectiveness of the ACG module is discussed in the ablation experiments of Section 5.4. This section addresses the effectiveness and usefulness of the ACG construction method used in this paper.

- Effectiveness: we verify how the ACG construction method used in this paper performs RCA.
- Usefulness: we verify if the ACG construction method used in this paper can correctly find the causal relationship between events.

To verify the effectiveness of ACG construction methods, we compare four methods: the PC algorithm (Kobayashi et al., 2022); the DirectLiNGAM algorithm (Wu et al., 2021); the Hawkes process algorithms under non-independently and identically distributed data, including THP (Cai et al., 2022) and TTHP proposed in this paper. The three baselines PC, DirectLiNGAM and THP are selected because they represent three major categories of current methods for anomaly correlation graph construction, which are respectively the constraint-based, function-based and Hawkes process-based methods. They are respectively used to learn the causal structure of abnormal events, calculate the weights of edges, and finally construct an anomaly correlation graph. Based on the causal edges and weight values in the ACG, the neighbor nodes with high causal correlation are selected as the aggregation targets of the central node by using the weight threshold sampling mechanism.

Table 4 summarizes the results that are obtained from the ACG construction by using different causal learning methods for root cause localization. It can be seen that our TTHP outperforms other methods in terms of performance indicators. The PC algorithm and DirectLiNGAM algorithm ignore the network topology behind alarms when learning the causal relationship of alarm events, and therefore they cannot achieve

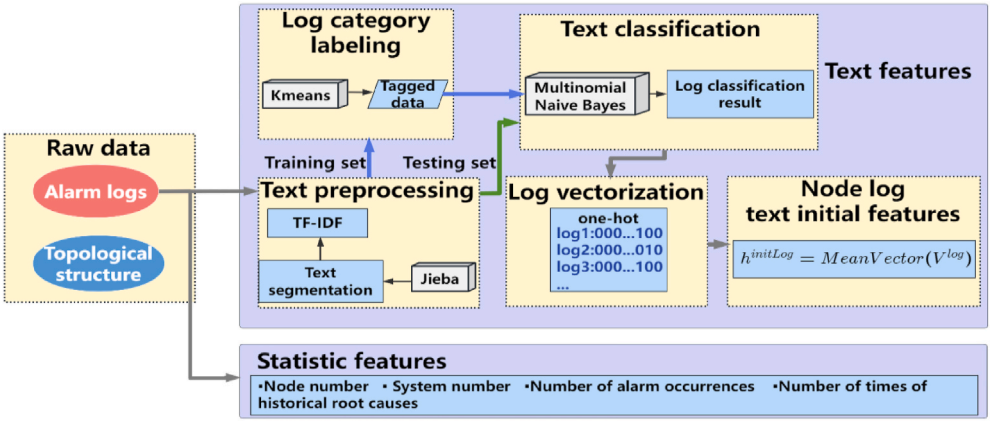


Fig. 10. Data processing in the Suning Dataset.

good causal learning results. The ACG constructed based on them only contains about 40 causal edges. If too few causal correlation edges are learned, the weight threshold sampling mechanism can filter many alarms that actually contain causal correlation as noise alarms when judging noise alarms, and this thus affects the extraction of node features. So, their accuracy rates are only 66.7% and 63.3%, respectively, and the probability that the top 5 nodes contain root causes is relatively low, only 83.3% and 75%. THP does not consider the influence of acquisition error in the real production environment when learning the causal structure, and this may lead to inaccurate causal correlations learned. TTHP adds the processing of time-delay, and as a result, the learned abnormal event correlations are richer, which can be effectively applied to node sampling. The accuracy of TTHP is increased by more than 5%, compared with THP. But, the recall, precision, F1, Pre@3 and Pre@5 of TTHP are increased by about 4%, compared with THP.

The Suning Dataset used for the RCA experiments does not contain causal markers. To verify the usefulness of our ACG construction method, we conduct experiment on Wireless Dataset with causal markers. The usefulness validation is to evaluate if the causal learning method can correctly find out the causal relationship. Precision, Recall, and F1 value are also used as evaluation metrics for causal learning. In order to distinguish the evaluation metrics used for RCA in Section 5.1.2, the evaluation metrics for causal learning are denoted by $Precision^c$, $Recall^c$, and $F1^c$, respectively. Here $Precision^c$ denotes the ratio of the number of correct edges in the causal structure of events learned by the algorithm to the number of edges in the learned causal structure; $Recall^c$ denotes the ratio of the number of correct edges in the causal structure of events learned by the algorithm to the number of edges in the original true causal graph; the $F1^c$ value is a measure of the combined precision and recall.

Table 5 shows the comparative results of causal learning methods that learn causal structure on the Wireless Dataset, with each metric presented as the mean ± standard deviation of 10 random splits. It is shown in Table 5 that TTHP performs better than other methods in three evaluation indicators. The PC algorithm and DirectLiNGAM algorithm are ineffective because they are not applicable to non-independently and identically distributed data. They respectively have the low recall^c values of 0.362 and 0.145, indicating that it is difficult for them to find correct causal relationship. THP cannot learn the causal mechanism behind the data with time error, so its effect is slightly inferior to TTHP, and its $F1^c$ and $Recall^c$ values are respectively about 10% lower than TTHP. In contrast, TTHP is able to obtain more causal relationships and can help to guide O&M personnel in fault localization and repair. Note that, however, the $Precision^c$ value of TTHP is similar to the $Precision^c$ value of THP.

Table 5
Comparison of causal learning methods on Wireless Dataset.

Method	F1^c	Recall^c	Precision^c
PC (Kobayashi et al., 2022)	0.423 ± 0.015	0.364 ± 0.012	0.512 ± 0.017
DirectLiNGAM (Wu et al., 2021)	0.166 ± 0.010	0.142 ± 0.008	0.195 ± 0.011
THP (Cai et al., 2022)	0.565 ± 0.020	0.493 ± 0.018	0.665 ± 0.022
TTHP (ours)	**0.664 ± 0.012**	**0.638 ± 0.014**	**0.683 ± 0.013**

5.3. Evaluation of the ACG-LGraphBM model

This section experimentally verifies the effectiveness of the ACG-LGraphBM model. We compare the proposed model with several relevant methods, including *Random Walk* (Li et al., 2022a; Ma et al., 2020; Zhang et al., 2020), *PageRank* (Wang et al., 2021), *MicroRCA* (Wu et al., 2020), and *GCN* (He and Zhao, 2020; Yan et al., 2022). The four baselines Random Walk, PageRank, MicroRCA and GCN are selected because they represent four major categories of current methods for identifying fault root causes, which are the unsupervised, Markov chain, Personalized PageRank, and supervised methods. The unsupervised method mainly performs root cause localization by learning the graph structure between nodes. Random Walk is a typical unsupervised RCA method. The main idea is to randomly access nodes in the graph and accumulate scores. The node with the most scores is the root cause node. The PageRank algorithm is used to measure the importance of nodes in the graph, which continuously iterates through probability transfer to make the probability of each node converge to a smooth distribution. The higher the probability value obtained, the more likely it is to be a root cause node. MicroRCA uses the Personalized PageRank (PPR) algorithm

Table 4
Comparison of causal learning methods for constructing ACG. (Each metric is expressed as the mean ± standard deviation of 10 random splits).

Method for causal learning to construct ACG	Number of edges in ACG	Recall	Precision	Accuracy	F1	Pre@3	Pre@5
PC (Kobayashi et al., 2022)	42	0.708 ± 0.141	0.680 ± 0.108	0.667 ± 0.122	0.694 ± 0.142	0.750 ± 0.145	0.833 ± 0.112
DirectLiNGAM (Wu et al., 2021)	40	0.667 ± 0.134	0.640 ± 0.143	0.633 ± 0.117	0.653 ± 0.132	0.750 ± 0.125	0.750 ± 0.131
THP (Cai et al., 2022)	202	0.875 ± 0.087	0.875 ± 0.097	0.867 ± 0.072	0.875 ± 0.092	0.917 ± 0.081	0.958 ± 0.085
TTHP (ours)	**223**	**0.917 ± 0.056**	**0.917 ± 0.079**	**0.933 ± 0.012**	**0.917 ± 0.021**	**0.958 ± 0.008**	**1 ± 0.000**

to identify root causes, which is a variant of the PageRank algorithm. Their major difference is that the PageRank algorithm randomly selects nodes for each jump, while PPR jumps based on the "preference" of nodes. In the experiment, the edge weight is used to calculate the anomaly score of nodes as the preference. The typical representative of supervised root cause methods is GCN. Both GCN and GAT aggregate the features of neighboring nodes to the central node. The difference is that GCN uses the Laplacian matrix, rather than the attention mechanism to assign different weights to neighbors. In addition, we consider three variants of the graph convolutional network. GCN-Linear employs a single fully connected layer with a sigmoid activation, assuming a linear decision boundary in the node representation space. GCN-MLP uses a lightweight two-layer multilayer perceptron (MLP) with ReLU activation and dropout, introducing non-linear transformation capability while maintaining computational efficiency. GCN-LGBM combines GCN with a tree-based LightGBM classifier, providing a direct comparison between neural network–based and tree-based classification strategies, and highlighting the potential benefits of modeling feature interactions through a tree-based approach. The experimental results of different methods are shown in Table 6, with each metric presented as the mean ± standard deviation of 10 random splits.

It is shown that the localization results of Random Walk, PageRank and MicroRCA are not ideal because they are almost unable to find the correct root cause node and their time consumptions are high also. The localization of a single sample by Random Walk, for example, takes more than 5s. Compared with unsupervised methods, supervised GCN, GCN-Linear, GCN-MLP, and GCN-LGBM models perform better in accuracy, and the proposed ACG-LGraphBM model performs best in accuracy. Notably, the vanilla GCN achieves moderate performance, but exhibits relatively large variance across different random splits, indicating sensitivity to data partitioning. When replacing the original prediction head with a Linear head, the performance degrades substantially and shows even higher variance, suggesting that a purely linear decision boundary is insufficient for effective root cause localization. The GCN-MLP variant improves upon the Linear head by introducing non-linear transformations and demonstrates more stable behavior. However, its overall performance still remains inferior to that of the vanilla GCN, indicating limited modeling capacity for complex feature interactions. In contrast, GCN-LGBM consistently outperforms GCN, GCN-Linear, and GCN-MLP across all evaluation metrics. This consistent advantage provides strong empirical evidence that incorporating a tree-based classifier is essential for effectively modeling non-linear interactions and heterogeneous feature distributions that cannot be adequately handled by standard neural prediction heads.

Note that the average run time of GCN, GCN-Linear, GCN-MLP and GCN-LGBM for a single sample is shorter compared to ACG-LGraphBM. Let n is the number of nodes, d is the feature dimension, and k is the average number of neighbors per node. Then the time complexity of GCN is O ($n{\cdot}k{\cdot}d$) and the time complexity of GAT is O ($n{\cdot}k + n{\cdot}k{\cdot}d$). Compared to GAT, the GCN model does not require the calculation of attention coefficients between node pairs, thus requiring the least computation time. In addition, the GCN-LGBM model eliminates the process of node sampling, requiring less computation time also. The ACG-LGraphBM model leverages GAT to capture adaptive attention across nodes, which incurs slightly higher computation compared to GCN and GCN-LGBM. However, its average runtime remains substantially lower than that of Random Walk, PageRank, and MicroRCA, demonstrating a favorable trade-off between computational efficiency and predictive performance. Moreover, the repeated random-split evaluation (mean ± standard deviation) shows that ACG-LGraphBM achieves not only superior accuracy but also stable performance across different splits, highlighting both effectiveness and robustness.

In order to further observe the effect of these models, we set $K = 1$, 3, 5, and 10 in the experiment and use the Pre@K index to evaluate each model. The results are shown in Fig. 11. We observe form Fig. 11 that the performance of the ACG-LGraphBM model proposed in this paper is superior to the traditional root cause localization methods in each value of K. The unsupervised Random Walk, PageRank and MicroRCA methods mainly utilize graph structure information in the root cause localization process, ignoring the characteristics of the nodes themselves, resulting in unsatisfactory results. The supervised GCN does not consider the different effects of neighbor nodes in the process of extracting node features, which can easily lead to inaccurate extracted features. Therefore, its performance is relatively poor, compared to the model proposed in this paper. By comparison, the ACG-LGraphBM model can better extract more important information from the topological structure and alarm information, thus performing better on the test set. The probability that the top three possible root nodes contain root causes (Pre@3) is more than 95%, and basically our model can accurately locate the root cause.

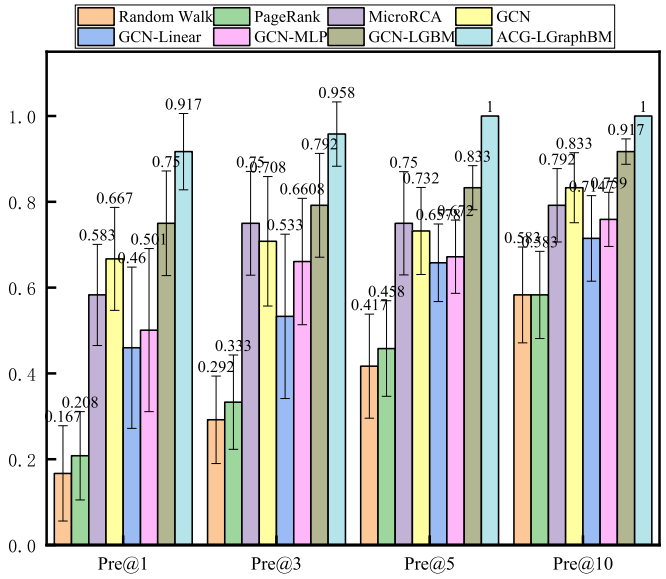


Fig. 11. Performance of the ACG-LGraphBM model and current typical RCA methods on Pre@K index.

Table 6
Performance comparison between the ACG-LGraphBM model and current typical RCA methods.

Method	Recall	Precision	Accuracy	F1	Average run time(s)
Random Walk (Li et al., 2022a; Ma et al., 2020; Zhang et al., 2020)	0.166 ± 0.122	0.161 ± 0.111	0.265 ± 0.115	0.162 ± 0.115	5.609
PageRank (Wang et al., 2021)	0.208 ± 0.114	0.208 ± 0.103	0.364 ± 0.108	0.205 ± 0.104	0.582
MicroRCA (Wu et al., 2020)	0.585 ± 0.123	0.583 ± 0.118	0.663 ± 0.121	0.581 ± 0.129	0.565
GCN (He and Zhao, 2020; Yan et al., 2022)	0.667 ± 0.121	0.616 ± 0.120	0.634 ± 0.122	0.645 ± 0.121	**0.202**
GCN-Linear	0.514 ± 0.204	0.460 ± 0.188	0.555 ± 0.141	0.562 ± 0.131	0.203
GCN-MLP	0.638 ± 0.126	0.501 ± 0.190	0.571 ± 0.083	0.628 ± 0.078	0.206
GCN-LGBM	0.745 ± 0.108	0.751 ± 0.122	0.767 ± 0.071	0.752 ± 0.074	0.207
ACG-LGraphBM (ours)	**0.916 ± 0.010**	**0.918 ± 0.029**	**0.932 ± 0.030**	**0.913 ± 0.009**	0.211

5.4. Parameter setting in model training

We discuss the parameter setting in model training, including: (1) *the weight threshold sampling mechanism in the ACG layer*; (2) *the number of heads (num_heads) of the multi-head attention mechanism in the GAT layer*; (3) *the learning rate, maximum number of leaves*; (4) *the number of segments of feature value discretization in the LGBM layer*.

Fig. 12 shows the relationship among feature aggregation time, root cause localization precision, and weight threshold, revealing how the threshold impacts on the aggregation time and precision. It is shown that, when the weight threshold is 0, no neighbor nodes are sampled and data noise affects the results, resulting in low precision of root cause localization and the long time for feature aggregation. It is also shown that, with the increase of the weight threshold, the number of aggregated neighbor nodes will decrease, and the number of connected edges of nodes involved in the calculation will also decrease accordingly. So, the feature aggregation time is reduced. Concerning root cause localization precision, with the increase of the weight threshold from 0 to 0.7, the precision is increased also. When the weight threshold is 0.7, the central node samples can aggregate the features of neighbor nodes whose causal correlation degree is no less than 0.7 and this can reduce the interference of noise, obtaining more than 90% precision. However, when the weight threshold continues to increase from 0.8 to 0.9, the precision decreases obviously because the number of sampling neighbors is greatly reduced at this time. This will lead to the loss of a lot of useful information and the decline of precision. For the experimental data used in the paper, it is demonstrated that setting the weight threshold as 0.7 can achieve the optimal effect.

Sometimes the training samples are incrementally updated and the cloud platform is dynamically deployed. For example, the sample increment reaches a threshold; the deployment structure changes greatly. When the data changes significantly, the model needs to be retrained so that the requirement of accurately locating the root cause can be met. The feature extraction module of the ACG-LGraphBM model uses the GAT model, where GAT uses inductive learning and can handle dynamic graphs. Fig. 13 shows the relationship between the number of iterations (i.e., *epoch*) and the time spent in LGBM classifier training and F1. In the experiment, the number of iterations is set to 140. At this time, the F1 value has converged to the optimum, and the time taken is 21.45s (the total number of alarm events in the training set is 59,394). This shows that the model proposed in this paper can complete retraining in a short period of time to cope with incremental data updates and dynamic deployment on cloud platforms. As shown in Fig. 14, the F1 score varies with the increase in training iterations and eventually stabilizes on both the training and test datasets.

Fig. 15 shows the relationship between the average run time, root cause localization precision, and attention head number of the GAT and ACG-LGraphBM models. It is shown that as the *num_heads* value increases, multiple attentions can learn richer information. So, the precision of the GAT and ACG-LGraphBM models is improved. When the *num_heads* value is 8, the optimal precision can be achieved. But, if we continue to increase the number of attention heads, it will not bring greater help to the precision improvement, and the running time required for the model to locate the root cause is longer. This is because, as the number of heads increases, the dimension of the features increases also, resulting in a longer running time. Therefore, to guarantee the precision and efficiency, the *num_heads* value in the experiment is set to 8. In the setting of LGBM layer parameters, the GAT-LGBM (without ACG) model is similar to the ACG-LGraphBM model. After testing, the learning rate is set to 0.03, the maximum number of leaves is set to 15, and the number of segments is set to 255.

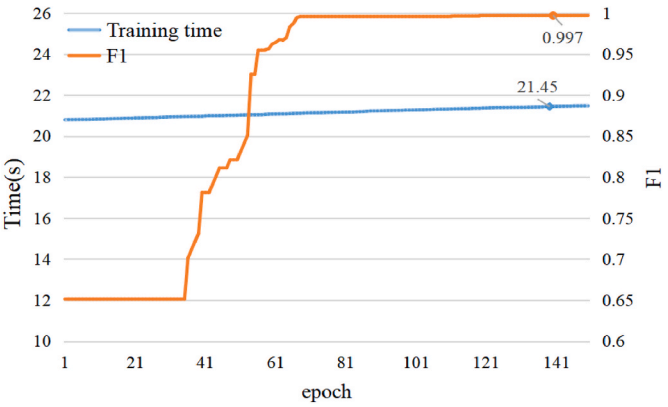


Fig. 13. Relationship between the number of iterations of classifier training and training time and F1 value.

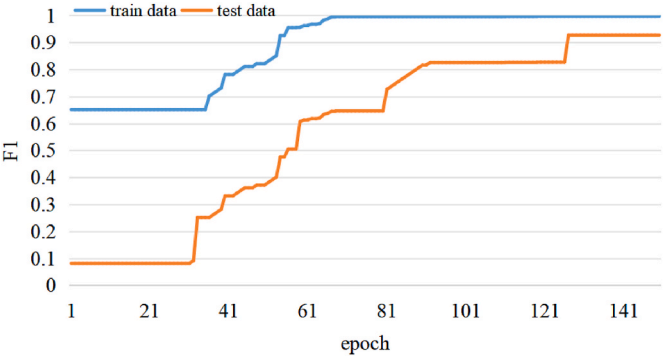


Fig. 14. F1 Score variation on training and test data.

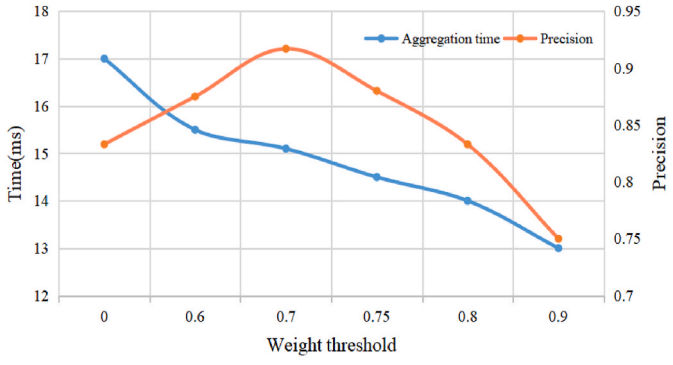


Fig. 12. Relationship between weight threshold and aggregate time and root cause localization precision.

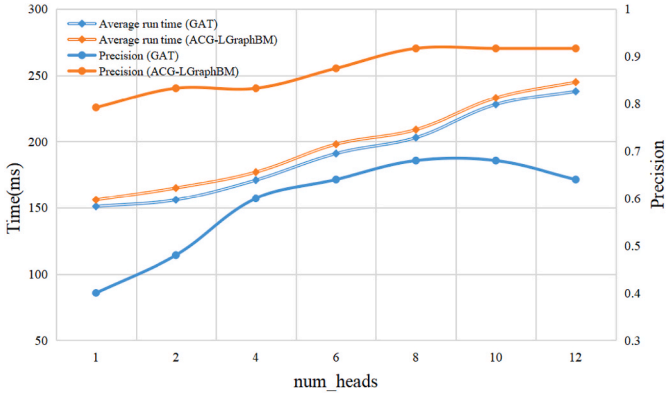


Fig. 15. Relationship between *num_heads* value and average run time and precision.

5.5. Ablation experiment of the ACG-LGraphBM model

We verify the effect of the ACG-LGraphBM model with ablation experiments by checking the use of an anomaly correlation graph (ACG) and LGBM in completing the final task of fault root cause analysis. To this end, we justify the effect of GAT by using the single GAT model. The single ACG or LGBM cannot complete the root cause analysis task independently. We apply the GAT-LGBM model to justify the effect of LGBM and apply the ACG-GAT model to justify the effect of ACG. We compare our ACG-LGraphBM model with the GAT model, the ACG-GAT model and the GAT-LGBM model. The performance results of the ablation experiment are shown in Table 7, with each metric presented as the mean ± standard deviation of 10 random splits.

It is shown in Table 7 that each component of the proposed model can improve the results to a certain extent. If the single model GAT (He and Zhao, 2020; Yan et al., 2022) is used only, the accuracy is 73.3% because deep learning models require a large amount of data and computational power, and it is very difficult to train stable GAT models. After integrating the GAT and LGBM models, the accuracy of the GAT-LGBM model is improved by 10%, compared to the GAT model. However, at this time, no judgment is made on noise alarms, and the positioning effect of the model is not accurate enough due to noise interference. After introducing the anomaly correlation graph and using the weight threshold sampling mechanism, noise interference can be reduced to a certain extent. Therefore, the accuracy of our model is 93.3%, the precision, recall and F1 values are all above 90%, and Pre@5 reaches 1. This fully demonstrates the role of each module (including ACG, GAT and LGBM) and the effectiveness of the ACG-LGraphBM model obtained by their fusion.

5.6. Expert study for interpretability evaluation

To quantitatively assess the interpretability of the proposed approach from a human expert perspective, we conducted a small-scale expert agreement study. Specifically, $N = 30$ representative fault alarm sequences were randomly sampled from the test set, covering common fault types such as CPU (Central Processing Unit) overload, memory anomalies, network failures, and application errors, as well as scenarios with varying system scales and complexities. We invited $M = 3$ engineers with practical experience in cloud platform operations and maintenance to participate in the annotation. For each sample, three experts independently completed the following two annotation tasks without access to the model outputs: (1) primary root cause node identification, where the expert selected the single node considered to be the most likely root cause; (2) root cause category labeling, by choosing one category from a predefined set (e.g., CPU, Memory, Network, Application). In order to evaluate the reliability of expert annotations, the expert decisions were treated as multi-rater discrete labels, and Fleiss' κ was computed at both the node level and the category level. Based on the expert annotations, a majority voting strategy was applied to obtain the expert consensus root cause node and category for each sample.

We compared the model outputs with the expert consensuses, in which the accuracies at the node level and category level were reported to measure the consistency between model explanations and human expert judgments. The quantitative results of the expert study are summarized in Table 8. It is shown in Table 8 that the inter-expert agreement reaches a Fleiss' κ of 0.762 at the node level and 0.813 at the category level, indicating a high degree of consistency among experts in identifying the primary root cause and its category. Furthermore, compared with the expert consensuses, the proposed method achieves an accuracy of 0.805 in primary root cause node identification and 0.925 in root cause category classification. These results demonstrate that the explanations produced by the proposed approach are largely aligned with expert judgments, providing quantitative evidence for its practical interpretability.

6. Conclusion

Based on the anomaly correlation graph, GAT and LGBM, we propose an ACG-LGraphBM model for RCA of faults in the cloud platforms with complex network topology and massive alarms. The model first learns the causal structure of topologically dependent abnormal alarm event sequences and constructs the anomaly correlation graph (ACG). Then, the weight threshold sampling mechanism is introduced to reduce noise interference. We use GAT algorithm to extract node features, so as to characterize the node information more accurately. Finally, the extracted features are used by LGBM classifier to obtain the root causes of faults. We also provide the interpretability analysis of the localization results. Being different from similar hybrid graph-based RCA frameworks, our model provides a combination pipeline to analyze root causes of cloud platform faults. Particularly, we propose the time-delay mechanism and a weight threshold sampling mechanism for ACG, improving the accuracy of root cause analysis. Experimental results show that the construction of anomaly correlation graph can make RCA results more precise, and the weight threshold sampling mechanism can play a certain role in denoising and improving the precision of root cause localization. Comparing the performance of our model with other typical RCA models, the proposed ACG-LGraphBM model achieves better results, with the root cause localization precision improvement of over 10%.

In our future work, we will further verify and improve the ACG-LGraphBM model from several crucial aspects such as: how to quantitatively evaluate its interpretability in the practical scenario; how to make cross-validation or multiple random splits for its robustness. In addition, the fault root cause analysis method in this paper is developed mainly for repetitive faults in a stable scenario because retraining the model is needed for the dynamic changes of cloud platform. However, in practical applications of fault root cause analysis, alarm data in logs, which are used to generate the ACG, may be sparse or change over time. For cases where specific alarm types are underrepresented or absent, in our future work, we would introduce large language models (LLMs) to enrich diversity and distribution of alarm types by supplementing new alarm data to logs. For a dynamic scenario that the ACG changes continuously, we will investigate how to conduct incremental or real-time updates to the ACG and how this continuous change affects the ACG's downstream processing. In addition, the ACG constructed in this

Table 8
Agreement statistics of the small-scale expert study.

Metric	Node-level	Category-level
Fleiss' κ (*inter-expert*)	0.762	0.813
Accuracy (*model* vs. *expert consensus*)	0.805	0.925

Table 7
Results of ACG-LGraphBM ablation experiment.

Model	Recall	Precision	Accuracy	F1	Pre@3	Pre@5	Average run time (s)
GAT (He and Zhao, 2020; Yan et al., 2022)	0.708 ± 0.121	0.680 ± 0.111	0.733 ± 0.124	0.694 ± 0.133	0.792 ± 0.102	0.875 ± 0.092	0.203
ACG-GAT (without LGBM)	0.750 ± 0.103	0.720 ± 0.099	0.767 ± 0.091	0.735 ± 0.083	0.833 ± 0.094	0.875 ± 0.086	0.206
GAT-LGBM (without ACG)	0.833 ± 0.081	0.833 ± 0.058	0.867 ± 0.062	0.833 ± 0.056	0.875 ± 0.084	0.917 ± 0.071	0.207
ACG-LGraphBM	0.916 ± 0.010	0.918 ± 0.029	0.932 ± 0.030	0.913 ± 0.009	0.958 ± 0.075	1 ± 0.000	0.211

paper cannot directly use the causal structure for fault classification, localization or further inference of fault resolution measures. We plan to combine knowledge graphs for fault root cause analysis in our future work. Furthermore, we intent to incorporate more systematic approaches to enhance interpretability, such as rule induction methods and frameworks like LIME (Local Interpretable Model-agnostic Explanations). This would provide users with a comprehensive guideline identifying the conditions leading to various anomaly states, as well as quantifying the associated risks under those conditions.

CRediT authorship contribution statement

Beijing Zhou: Data curation, Validation, Visualization. **Bingyan Chen:** Writing – original draft. **Li Yan:** Writing – review & editing. **Xinyu Liang:** Validation. **Zongmin Ma:** Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Appendix. Abnormal event types in the dataset

All alarm events in the datasets are labeled into 29 types, as shown in Table 9.

Table 9
Description of abnormal event types.

Abnormal event type	Abnormal event type description
v_0	The request delay exceeds the threshold.
v_1	The idle CPU lasts for n minutes less than the threshold.
v_2	The log reports an error: the connection cannot be obtained.
v_3	The average transmit traffic for eth0 exceeds the threshold of n minutes.
v_4	Port communication anomaly.
v_5	The number of HTTP slow responses is greater than the threshold.
v_6	URL access failed.
v_7	The application stops running.
v_8	The ping delay duration exceeds the threshold for n minutes.
v_9	The average heap memory usage exceeds the threshold.
v_{10}	The I/O waiting load lasts for n minutes exceeding the threshold.
v_{11}	The log generates error messages.
v_{12}	The I/O waiting load is greater than the threshold.
v_{13}	The free swap space is smaller than the threshold.
v_{14}	The average received traffic of network card eth0 exceeds the threshold for n minutes.
v_{15}	Hard disk Slot00 status is failed.
v_{16}	FullGC count greater than threshold.
v_{17}	The number of connections to JBOSS n ports is greater than the threshold.
v_{18}	SDD IO usage lasts n minutes greater than the threshold.
v_{19}	CPU Steal Time lasts n minutes and exceeds the threshold.
v_{20}	The ping packet loss rate is 100%, and the server breaks down.
v_{21}	FullGC average time is greater than the threshold.
v_{22}	The remaining space of the partition is less than the threshold.
v_{23}	The current value of the ping packet loss rate is x, which lasts for n minutes on average greater than the threshold.
v_{24}	Network interface card traffic unknown.
v_{25}	There is "no route to host" information in the log.
v_{26}	The number of connections to the Nginx80 port is greater than the threshold.
v_{27}	The average value of the traffic sent by eth0 for n minutes is greater than the threshold.
v_{28}	There is "OutOfMemoryError" information in the log.

Data availability

Data will be made available on request.

References

Alinezhad, H.S., Roohi, M.H., Chen, T., 2022. A review of alarm root cause analysis in process industries: common methods, recent research status and challenges. Chem. Eng. Res. Des. 188, 846–860.

Arunthavanathan, R., Khan, F., Ahmed, S., Imtiaz, S., 2022. Autonomous fault diagnosis and root cause analysis for the processing system using one-class SVM and NN permutation algorithm. Ind. Eng. Chem. Res. 61 (3), 1408–1422.

Böhmer, K., Rinderle-Ma, S., 2020. Mining association rules for anomaly detection in dynamic process runtime behavior and explaining the root cause to users. Inf. Syst. 90, 101438.

Cai, R., Wu, S., Qiao, J., Hao, Z., Zhang, K., Zhang, X., 2022. THPs: topological hawkes processes for learning causal structure on event sequences. IEEE Transact. Neural Networks Learn. Syst. 1–15.

Dehbozorgi, M.R., Rastegar, M., Dabbaghjamanesh, M., 2020. Decision tree-based classifiers for root-cause detection of equipment-related distribution power system outages. IET Gener. Transm. Distrib. 14 (24), 5809–5815.

Deng, D., Chen, X., Zhang, R., Lei, Z., Wang, X., Zhou, F., 2021. XGraphBoost: extracting graph neural network-based features for a better prediction of molecular properties. J. Chem. Inf. Model. 61 (6), 2697–2705.

Ding, Y., Teng, F., Zhang, P., Huo, X., Sun, Q., Qi, Y., 2021. Research on text information mining technology of substation inspection based on improved jieba. In: Proceedings of the 2021 International Conference on Wireless Communications and Smart Grid, pp. 561–564.

Gaita, A., Buzo, A., David, E., Cucu, H., Pelz, G., 2022. A machine learning based wafer test ranking for root cause analysis. In: Proceedings of the 2022 International Symposium ELMAR, pp. 45–48.

Ghazal, T.M., Hussain, M.Z., Said, R.A., Nadeem, A., Hasan, M.K., Ahmad, M., et al., 2021. Performances of K-Means clustering algorithm with different distance metrics. Intell. Autom. Soft Comput. 30 (2), 735–742.

Hamilton, W., Ying, Z., Leskovec, J., 2017. Inductive representation learning on large graphs. Adv. Neural Inf. Process. Syst. 30.

He, J., Zhao, H., 2020. Fault diagnosis and location based on graph neural network in telecom networks. In: Proceedings of the 2020 International Conference on Networking and Network Applications. IEEE, pp. 304–309.

Hu, W., Pu, J., Tang, X., Liu, X., 2025. Contrastive enhanced knowledge distillation for learning MLPs on GNNs. Neural Process. Lett. 57 (4), 68.

Hu, W., Yang, G., Li, Y., Cao, W., Wu, M., 2023. Root cause identification of industrial alarm floods using word embedding and few-shot learning. IEEE Trans. Ind. Inf. 1–11.

Jiang, N., Jie, W., Li, J., Liu, X., Jin, D., 2022. GATrust: a multi-aspect graph attention network model for trust assessment in OSNs. IEEE Trans. Knowl. Data Eng. 4 (6), 2962–2974.
Jiang, W., Bai, Y., 2023. APGNN: alarm propagation graph neural network for fault detection and alarm root cause analysis. Comput. Netw. 220, 109485.
Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., et al., 2017. LightGBM: a highly efficient gradient boosting decision tree. Adv. Neural Inf. Process. Syst. 3146–315430.
Kobayashi, S., Shima, K., Cho, K., Akashi, O., Fukuda, K., 2022. Comparative causal analysis of network log data in two large ISPs. In: Proceedings of the NOMS 2022-2022 IEEE/IFIP Network Operations and Management Symposium, pp. 1–6.
Lewis, E., Mohler, G., 2011. A nonparametric EM algorithm for multiscale hawkes processes. J. Nonparametric Statistics 1 (1), 1–20.
Li, Z., Tu, Y., Ma, Z., 2022b. Root cause analysis of anomalies based on graph convolutional neural network. Int. J. Software Eng. Knowl. Eng. 32 (8), 1155–1177.
Li, Q., Zhang, Y., Hua, X., Zhu, L., Lei, Z., 2022a. Structure learning-based unsupervised root cause diagnosis for radio access networks. Electron. Lett. 58 (10), 414–416.
Li, Z., Zhao, N., Li, M., Lu, X., Wang, L., Chang, D., et al., 2022c. Actionable and interpretable fault localization for recurring failures in online service systems. In: Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, pp. 996–1008.
Liu, D., He, C., Peng, X., Lin, F., Zhang, C., Gong, S., et al., 2021. MicroHECL: high-efficient root cause localization in large-scale microservice systems. In: Proceedings of the 2021 IEEE/ACM 43rd International Conference on Software Engineering: Software Engineering in Practice, pp. 338–347.
Liu, G., Yan, Y., Cai, J., et al., 2025. GCD: graph contrastive denoising module for GNNs in EEG classification. Expert Syst. Appl. 265, 126013.
Ma, M., Xu, J., Wang, Y., Chen, P., Zhang, Z., Wang, P., 2020. Automap: diagnose your microservice-based web applications automatically. In: Proceedings of the Web Conference 2020, pp. 246–258.
Marques-Silva, J., Gerspacher, T., Cooper, M., Ignatiev, A., Narodytska, N., 2020. Explaining naive bayes and other linear classifiers with polynomial time and delay. Adv. Neural Inf. Process. Syst. 33, 20590–20600.
Meng, Y., Zhang, S., Sun, Y., Zhang, R., Hu, Z., Zhang, Y., et al., 2020. Localizing failure root causes in a microservice through causality inference. In: Proceedings of the 2020 IEEE/ACM 28th International Symposium on Quality of Service. IEEE, pp. 1–10.
Ni, Z., Liu, H., Chen, Y., Wu, D., 2019. Research and implementation of a method for web log analysis template extraction. Procedia Comput. Sci. 162, 673–681.
Notaro, P., Cardoso, J., Gerndt, M., 2021. A survey of AIOps methods for failure management. ACM Trans. Intell. Syst. Technol. 12 (6), 1–45.
Parvez, M.R., Hu, W., Chen, T., 2022. Real-time pattern matching and ranking for early prediction of industrial alarm floods. Control Eng. Pract. 120, 105004.
Qaiser, S., Ali, R., 2018. Text mining: use of TF-IDF to examine the relevance of words to documents. Int. J. Comput. Appl. 181 (1), 25–29.
Qiu, J., Du, Q., Yin, K., Zhang, S.L., Qian, C., 2020. A causality mining and knowledge graph based method of root cause diagnosis for performance anomaly in cloud applications. Appl. Sci. 10 (6), 2166–2185.
Qu, Y., Montzka, C., Vereecken, H., 2021. Causation discovery of weather and vegetation condition on global wildfire using the PCMCI approach. In: Proceedings of the 2021 IEEE International Geoscience and Remote Sensing Symposium. IEEE, pp. 8644–8647.
Shen, S., Zhang, J., Huang, D., Xiao, J., 2020. Evolving from traditional systems to AIOps: Design, implementation and measurements. In: Proceedings of the 2020 IEEE International Conference on Advances in Electrical Engineering and Computer Applications. IEEE, pp. 276–280.
Soldani, J., Brogi, A., 2022. Anomaly detection and failure root cause analysis in (micro) service-based cloud applications: a survey. ACM Comput. Surv. 55 (3), 1–39.
Sun, Y., Ding, S., Zhang, Z., Jia, W., 2021. An improved grid search algorithm to optimize SVR for prediction. Soft Comput. 25, 5633–5644.
Ta, P.B., Duong, P.H., Kim, Y., 2023. A-RCL: an adaptive incident prediction and automated root cause localization system for cloud environment. In: Proceedings of the 2023 IEEE/IFIP Network Operations and Management Symposium. IEEE, pp. 1–5.
Wan, Z., Liu, X., Wang, B., et al., 2024. Spatio-temporal contrastive learning-enhanced GNNs for session-based recommendation. ACM Trans. Inf. Syst. 42 (2), 58:1–58:26.
Wang, H., Wu, Z., Jiang, H., Huang, Y., Wang, J., Kopru, S., Xie, T., 2021. Groot: an event-graph-based approach for root cause analysis in industrial settings. In: Proceedings of the 2021 36th IEEE/ACM International Conference on Automated Software Engineering. IEEE, pp. 419–429.
Wu, L., Tordsson, J., Bogatinovski, J., Elmroth, E., Kao, O., 2021. Microdiag: fine-grained performance diagnosis for microservice systems. In: Proceedings of the 2021 IEEE/ACM International Workshop on Cloud Intelligence. IEEE, pp. 31–36.
Wu, L., Tordsson, J., Elmroth, E., Kao, O., 2020. MicroRCA: root cause localization of performance issues in microservices. In: Proceedings of the NOMS 2020-2020 IEEE/IFIP Network Operations and Management Symposium. IEEE, pp. 1–9.
Xia, L., Huang, C., Huang, C., Lin, K., Yu, T., Kao, B., 2023. Automated self-supervised learning for recommendation. In: *Proceedings of the ACM Web Conference* 2023, pp. 992–1002.
Yan, S., Shan, C., Yang, W., Xu, B., Li, D., Qiu, L., et al., 2022. CMMD: cross-metric multi-dimensional root cause analysis. In: Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, pp. 4310–4320.
Yao, Z., Pei, C., Chen, W., Wang, H., Su, L., Jiang, H., et al., 2024. Chain-of-Event: interpretable root cause analysis for microservices through automatically learning weighted event causal graph. In: Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering, pp. 50–61.
You, L., Jiang, H., Hu, J., Chang, C.H., Chen, L., Cui, X., Zhao, M., 2022. GPU-accelerated faster mean shift with euclidean distance metrics. In: Proceedings of the 2022 IEEE 46th Annual Computers, Software, and Applications Conference. IEEE, pp. 211–216.
Zhao, J., Wen, Q., Ju, M., Zhang, C., Ye, Y., 2023. Self-supervised graph structure refinement for graph neural networks. In: Proceedings of the Sixteenth ACM International Conference on Web Search and Data Mining, pp. 159–167.
Zhang, L., Zhao, J., Zhang, M., 2020. Root cause analysis of concurrent alarms based on random walk over anomaly propagation graph. In: Proceedings of the 2020 IEEE International Conference on Networking, Sensing and Control. IEEE, pp. 1–6.
Zhang, Y., Guan, Z., Qian, H., Xu, L., Liu, H., Wen, Q., et al., 2021a. CloudRCA: a root cause analysis framework for cloud Computing platforms. In: Proceedings of the 30th ACM International Conference on Information & Knowledge Management, pp. 4373–4382.
Zhang, S., Tong, H., Xu, J., Maciejewski, R., 2019. Graph convolutional networks: a comprehensive review. Computational Social Networks 6 (1), 1–23.
Zhang, Y., Wang, X., Shi, C., Jiang, X., Ye, Y., 2021b. Hyperbolic graph attention network. IEEE Trans. Big Data 8 (6), 1690–1701.
Zheng, L., Chen, Z., He, J., Chen, H., 2024. MULAN: multi-Modal causal structure learning and root cause analysis for microservice systems. In: Proceedings of the ACM on Web Conference 2024, pp. 4107–4116.